



UNIVERSIDADE ESTADUAL DE CAMPINAS

Faculdade de Engenharia Mecânica

Programa de Pós-graduação em Engenharia Mecânica



Luiz Claudio Marangoni de Oliveira

RELATÓRIO FINAL DE PÓS-DOCTORADO

Campinas - SP

2025

Luiz Claudio Marangoni de Oliveira

**Robótica Educacional Avançada: Desenvolvimento de
Ambiente Virtual com Ferramentas de Código Aberto
ROS/Gazebo/MoveIt para o ensino de robótica**

**Relatório Científico de Encerramento do Programa de Pós-Doutorado apresentado ao
Departamento de Sistemas Integrados da Faculdade de Engenharia Mecânica da
Universidade Estadual de Campinas**

Período: 01/08/2024 a 01/08/2025

Supervisor: Prof. Dr. Paulo Roberto Gardel Kurka

Campinas - SP

2025

1. RESUMO

A robótica está cada vez mais presente no cotidiano. É possível encontrar aplicações comerciais de robótica em aspiradores, veículos autônomos, assistentes domésticos, sistemas de vigilância e monitoramento, entre outros. Da mesma forma, as aplicações industriais dos sistemas robóticos têm se popularizado e se tornado cada vez mais comuns, como nos sistemas complexos utilizados na indústria 4.0, no setor automotivo, eletroeletrônico, na agricultura, em armazéns e depósitos, na medicina e em muitas outras áreas. O crescimento da infraestrutura de Internet das Coisas (IoT) tem possibilitado a integração desses dispositivos robóticos em redes e o desenvolvimento de algoritmos que adicionam inteligência a esses sistemas, permitindo a tomada de decisões em tempo real a partir de dados de sensores e informações armazenadas na nuvem, os chamados sistemas robóticos inteligentes.

Tradicionalmente, instituições de ensino de automação têm investido em laboratórios com sistemas robóticos físicos tais como, células de manufatura automatizadas, sistemas inteligentes, integração de dados e indústria 4.0, dentre outros. Esses ambientes possibilitam a capacitação de profissionais e o desenvolvimento de pesquisas em sistemas semelhantes aos presentes na indústria. Entretanto, os elevados custos de implantação e manutenção dificultam sua adoção por muitas instituições. Além disso, os avanços tecnológicos frequentes nesta área demandam atualização constante de equipamentos e sistemas para não ficarem defasados em relação à realidade da indústria. Essa situação impõe às instituições de ensino na área o desafio da manutenção constante destes ambientes e faz com que a formação adequada dos futuros engenheiros e profissionais seja crítica.

Nesse cenário, os MOOCs (*Massive Open Online Courses*) são uma das soluções adotadas por conciliarem formação teórica e prática, flexibilidade de conteúdos e possibilitarem uma formação específica em nível superior totalmente personalizada para uma grande quantidade de estudantes. Esses cursos só se tornaram viáveis graças ao desenvolvimento de ambientes e/ou plataformas virtuais de aprendizagem focadas em sistemas robóticos.

Essas plataformas virtuais oferecem ambientes acessíveis de forma segura e a possibilidade de experimentação sem a necessidade de instalações específicas, acesso a uma grande variedade de modelos de robôs que podem ser facilmente integrados aos ambientes virtuais, a criação de cenários customizados conforme as demandas da formação desejada, desde conceitos básicos até tópicos avançados, adaptando-se às necessidades dos usuários. Para o desenvolvimento desses ambientes virtuais, nota-se uma tendência de utilização de ferramentas de código aberto, como o ROS (Robot Operating System), o simulador Gazebo, entre outros.

Neste projeto foram realizados estudos sobre algumas das principais plataformas e ambientes virtuais abertos para o ensino de robótica e foi desenvolvida uma plataforma com um ambiente virtual

aberto e modular com um conjunto inicial de exemplos para o ensino de robótica. O **LabViR** - Laboratório Virtual de Robótica - é um ambiente desenvolvido para o ensino e pesquisa de robótica baseado em ROS com aplicações containerizadas baseadas no Docker flexível e aberta que disponibiliza uma série de cursos em formato modular que podem ser acessados localmente ou remotamente através de acesso a nuvem. Esta plataforma pode ser utilizada como complemento aos caros e, muitas vezes, inexistentes ambientes físicos para o ensino de robótica em nível de graduação.

Palavras-chave: Ensino de Robótica, educação em nível superior, inovação em educação, Robot Operating System (ROS), Gazebo, MoveIt, Ambientes virtuais, Docker, Manipuladores robóticos

2. INTRODUÇÃO

STEM é o acrônimo em inglês para Ciência, Tecnologia, Engenharia e Matemática. Nos últimos anos esta área da educação tem crescido bastante em razão do aumento da demanda de profissionais nas áreas de computação, matemática, engenharia, arquitetura e ciências. Como exemplo, nos Estados Unidos em 2021 haviam de 11 milhões de profissionais nestas áreas e projeta-se um crescimento de 11% neste total até 2031, mais do que o dobro do que em outras áreas (KRUTSCH; RODERICK, 2022)

A robótica está intrinsecamente ligada à educação em STEM e pode ser descrita como uma disciplina que aborda uma classe de sistemas mecatrônicos chamados robôs, capazes de realizar diversas aplicações industriais, científicas e comerciais. Essa disciplina tornou-se uma parte crítica dos sistemas mecatrônicos industriais nos últimos anos. É por isso que programas de graduação profissional, como engenharia mecatrônica, engenharia eletrônica e engenharia de computação, tornaram-se fundamentais para o ensino de robótica e incorporam muitos dos conceitos básicos de STEM (KHINE, 2017).

Devido às constantes mudanças nas tecnologias e na robótica em diferentes indústrias, preparar adequadamente os futuros engenheiros é crucial. Para realizar isso, os professores devem desenvolver e implementar maneiras inovadoras de ensinar conceitos e usar novas tecnologias para otimizar o aprendizado (VERNER; CUPERMAN; REITMAN, 2021).

Neste sentido, uma inovação importante surgida nos últimos tempos foi o *Robot Operating System* (ROS) que se tornou uma estrutura de desenvolvimento de software padrão na robótica. Isso se deve, em grande parte, à sua flexibilidade, pois não impõe muitas restrições aos desenvolvedores. Com o ROS, é possível acessar uma vasta coleção de bibliotecas de software, construir códigos modulares na linguagem de programação de sua escolha e integrar dispositivos de diferentes fabricantes. Aprender robótica utilizando o ROS garante que as habilidades adquiridas sejam aplicáveis no mercado de trabalho, facilitando a transição do contexto acadêmico para o ambiente profissional (MICHIELETTTO *et al.*, 2014; QUIGLEY *et al.*, [S.d.])

Entretanto, segundo (KRUMINS *et al.*, 2024), aprender robótica com ROS é desafiador, pois essa tecnologia assume dois papéis simultaneamente: 1) o próprio conteúdo de estudo e 2) o meio pelo qual a experiência de aprendizado é mediada (BOWER, 2019)

Outro aspecto desafiador no ensino de robótica é que para efetividade do aprendizado espera-se que o aluno tenha à disposição um ambiente de desenvolvimento com um robô específico e seu framework de software funcionais. Mesmo utilizando-se o ROS, isso constitui um desafio pois exige uma configuração inicial considerável, seja por parte do aluno ou instrutor. Esse processo geralmente

envolve a instalação do sistema operacional Linux — muitas vezes inédito para o aluno — além dos pacotes necessários do ROS e de softwares específicos dos robôs (PIETRZIK; CHANDRASEKARAN, 2019). Mesmo com tutoriais bem estruturados, essa etapa pode ser intimidante e desmotivadora, especialmente para iniciantes. Ademais, a instalação pode demandar permissões de administrador na máquina do aluno e qualquer erro no processo pode resultar em problemas no sistema ou até na perda de dados pessoais.

Uma maneira inovadora de ensinar conceitos de robótica e oferecer atividades práticas é utilizando plataformas de robótica. No entanto, essas plataformas apresentam vários desafios em termos de acessibilidade, custo e flexibilidade. Uma plataforma acessível é útil para obter ou substituir componentes, se necessário, e auxiliar os alunos na educação. Da mesma forma, a flexibilidade dessas ferramentas educacionais permite sua adaptação a diferentes metodologias de ensino e currículos. Finalmente, o custo é uma consideração significativa. Em muitos casos, não é viável para instituições educacionais com muitos alunos investirem em mais de uma plataforma para atender às necessidades de inúmeras turmas (KARALEKAS; VOLOGIANNIDIS; KALOMIROS, 2020b)

Neste contexto, os laboratórios remotos surgem como uma solução tecnológica capaz de oferecer tanto o acesso a robôs físicos quanto a ambientes de simulação, minimizando a necessidade de configuração local e maximizando a personalização do aprendizado (GUIMARÃES *et al.*, 2011). Eles vêm sendo cada vez mais reconhecidos como ferramentas valiosas na educação, especialmente nas áreas de engenharia e computação (MARTIN *et al.*, 2021) Aplicações baseadas na web permitem que os alunos programem projetos robóticos sem precisar executar software especial em seus computadores ou ter o robô fisicamente perto deles (ROLDÁN-´ ALVAREZ *et al.*, 2022)

Por conta disso, a educação a distância em robótica baseada na web é um campo crescente da educação em engenharia. O desenvolvimento de ferramentas capazes de realizar simulações em diferentes ambientes, utilizando diferentes tipos de robôs e que permitam o estudo de diferentes algoritmos para resolução de problemas tem sido objeto de vários estudos na área tais como (ANAND *et al.*, 2021; CASAÑ *et al.*, 2015; DOS SANTOS LOPES *et al.*, 2017; KRUMINS *et al.*, 2024; KULICH *et al.*, 2013; PETROVI; BALOGH, 2012; PITZER *et al.*, 2012; TELLEZ, 2017; VENTUZELOS; LEÃO; SOUSA, 2023; WIEDMEYER *et al.*, 2019)

No universo da robótica com ROS existem diversos recursos disponíveis para diferentes áreas da robótica. Dentre estes um dos mais conhecidos é o MoveIt! (SUCAN; CHITTA, 2018) uma plataforma de software de código aberto para planejamento de movimento de robôs que integra diversos componentes cruciais para o planejamento de movimento, como a cinemática direta e inversa, a geração de trajetórias, a evasão de colisões e a percepção ambiental. Esta integração facilita a

criação de soluções completas de robótica, reduzindo significativamente o tempo e o esforço necessários para desenvolver algoritmos personalizados (CHITTA; SUCAN; COUSINS, 2012).

Com o objetivo de proporcionar aos estudantes uma aprendizagem mais realista e envolvente e de minimizar as dificuldades nas configurações iniciais típicas de quem ingressa no universo da robótica com o ROS, este trabalho propõe a criação de um sistema web de código aberto e flexível. Este sistema permite que estudantes e professores acessem ambientes de desenvolvimento prontos para uso para programação de robôs utilizando o ROS de forma interativa, sem a necessidade de instalação de softwares localmente. Na plataforma são disponibilizados vários tutoriais e atividades sobre o ROS com exemplos de manipuladores robóticos comerciais e uso do MoveIt!. Adicionalmente são disponibilizados os ambientes virtualizados e pré-configurados para download e utilização no computador local, o que padroniza e agiliza no desenvolvimento de projetos e pesquisas que utilizem o ROS como base.

Com o objetivo de promover um aprendizado mais envolvente e eliminar as barreiras associadas à configuração inicial enfrentadas por quem ingressa no mundo do ROS, este trabalho propõe um sistema web de código aberto. Este sistema permite que os alunos acessem ambientes de desenvolvimento prontos para uso, nos quais podem programar robôs ROS — tanto simulados quanto físicos — de forma interativa. Assim, os estudantes podem iniciar sua jornada no aprendizado da robótica sem precisar instalar softwares ou adquirir robôs, utilizando um ambiente de desenvolvimento que reflete as práticas reais do mercado.

3. JUSTIFICATIVA

A crescente adoção dos sistemas robóticos nos mais diversificados ramos que vão desde aplicações domésticas, veículos autônomos, sistemas de vigilância e monitoramento, agricultura robotizada, medicina, logística, indústria aeroespacial, até os sistemas industriais da indústria 4.0, a velocidade do surgimento de novas tecnologias e dispositivos agregado ao uso onipresente dos sistemas conectados em redes de alta velocidade representam um desafio ao ensino de robótica em instituições do mundo inteiro. Tradicionalmente, as instituições de ensino têm investido em laboratórios com sistemas robóticos e sistemas inteligentes que possibilitam a capacitação de profissionais e a realização de pesquisas em ambientes semelhantes aos existentes na indústria. Entretanto, os elevados custos de implantação e manutenção dificultam sua adoção por muitas instituições. Além disso, os avanços tecnológicos frequentes nesta área demandam atualização constante de equipamentos e sistemas para não ficarem defasados em relação à realidade da indústria. Essa situação impõe às instituições de ensino na área o desafio da atualização e manutenção constante destes ambientes e faz com que a formação adequada dos futuros engenheiros e profissionais seja crítica.

Os laboratórios virtuais surgem como uma solução tecnológica capaz de oferecer acesso a robôs físicos e à ambientes de simulação e permite a ampliação da variedade de dispositivos robóticos que pode ser programada, a atualização dos sistemas robóticos para uso das novas tecnologias e uma transição facilitada dos dispositivos simulados para os existentes em campo. Baseados no ROS tais ambientes permitem o uso de inúmeras bibliotecas e plataformas integradas com as mais diversas finalidades, como Gazebo, RViz, MoveIt! e outras que possibilitam a criação e adaptação de cenários personalizados para estudos e pesquisas em robótica.

A incorporação de ambiente virtuais traz muitos benefícios, mas também alguns desafios:

- Necessidade de conhecimentos e prática de programação orientada a objetos em Python e/ou C++;
- Configuração e utilização de ambientes virtualizados baseados em Docker, Binder, Jupyter e outros;
- Utilização da linguagem de marcação XML nos arquivos de lançamento do ROS 1, descrição de robôs e outros;
- Conhecimento dos Sistemas de build utilizados no ROS (CMake e Catkin no ROS 1, Colcon no ROS 2), função os arquivos CMakeLists e package.xml e de compiladores como o Make e o CMake.

Tais conhecimentos, embora cada vez mais necessários, nem sempre são comuns aos estudantes ou pesquisadores da área de engenharia relacionados à robótica e representam um obstáculo que muitas vezes pode desanimar os estudantes iniciantes na área.

4. OBJETIVO GERAL

Este projeto propõe o desenvolvimento de uma plataforma online, baseada em tecnologias *open source*, voltada ao ensino e pesquisa em robótica no ensino superior. A plataforma atuará como uma camada de abstração de software, permitindo que desenvolvedores com conhecimentos básicos em computação e programação criem aplicações robóticas de forma simplificada. Estruturada em módulos temáticos, ela oferecerá aulas, tutoriais, atividades e projetos práticos, organizados em repositórios independentes que podem ser executados localmente ou acessados remotamente via navegador, eliminando a necessidade de instalação de sistemas operacionais, versões específicas do ROS e pacotes adicionais. Todo o ambiente será disponibilizado de forma containerizada, utilizando Docker, garantindo isolamento, segurança e fácil replicação dos setups. A utilização do ROS como base torna o sistema altamente flexível e expansível, permitindo a integração de diferentes robôs, sensores e atuadores, bem como a inclusão de modelos personalizados desenvolvidos em ferramentas como OnShape ou Blender. O projeto também visa possibilitar o desenvolvimento e teste de algoritmos avançados nas áreas de visão computacional, aprendizado de máquina, SLAM, entre outros. Dessa

forma, a proposta busca criar um ambiente colaborativo que favoreça a utilização, o compartilhamento e a disseminação de conteúdos, atividades e projetos entre grupos de pesquisa, ampliando o acesso a recursos para desenvolvimento e inovação em robótica.

4.1. Objetivos Específicos

- Estudo das ferramentas de código aberto adotadas no projeto – framework ROS, simulador Gazebo, framework de planejamento de trajetórias MoveIt.
- Análise sistemática de plataformas e ambientes virtuais para o ensino de robótica – UniRobotics, Robotics Academy e EUROPA para o estabelecimento de referenciais práticos e definição da abordagem que será adotada, modelos e cenários que serão inicialmente incorporados ao ambiente que será proposto.
- Elaboração de conjunto de aplicações baseadas em ROS com o Gazebo e MoveIt para demonstração de conceitos fundamentais de robótica – cinemática, dinâmica e controle de sistemas robóticos.
- Codificação do ambiente de ensino com definição dos cenários virtuais, incorporação dos exemplos e exercícios previamente desenvolvidos, elaboração de guia para utilização e instalação das ferramentas. Disponibilização do material em repositórios abertos tais como Github ou similares.

5. MATERIAIS E MÉTODOS

O presente projeto resultou no desenvolvimento de uma plataforma online de ensino e pesquisa em robótica, baseada em tecnologias de código aberto amplamente consolidadas no meio acadêmico e industrial, como o ROS, Gazebo e MoveIt.

O ambiente foi concebido para atuar como uma camada de abstração de software, que simplifica o acesso às ferramentas profissionais utilizadas no desenvolvimento de aplicações robóticas. Com isso, busca-se reduzir as barreiras iniciais frequentemente encontradas no ensino de robótica, relacionadas principalmente à configuração de ambientes, dependências de sistemas operacionais e dificuldades técnicas associadas à instalação de pacotes do ROS e seus simuladores.

A plataforma é composta por um conjunto de ambientes virtualizados, organizados em módulos temáticos. Cada módulo aborda conceitos específicos da robótica, como controle de robôs móveis, manipulação robótica, planejamento de movimento e integração de sensores. Os usuários podem acessar esses ambientes diretamente na nuvem, via navegador, por meio de ferramentas como o GitHub Codespaces ou instanciar os ambientes localmente, utilizando containers Docker. Os módulos

incluem exemplos práticos, tutoriais interativos, notebooks em JupyterLab (PROJECT JUPYTER, 2025), além de atividades guiadas com suporte à simulação no Gazebo e ao planejamento de movimento via MoveIt.

Além disso, todos os materiais didáticos, modelos de robôs, cenários e códigos-fonte estão organizados em repositórios públicos no GitHub, garantindo tanto a transparência quanto a possibilidade de expansão e customização por parte da comunidade acadêmica.

Na sequência deste documento apresentaremos, de forma resumida, as principais ferramentas utilizadas no desenvolvimento do projeto e a metodologia utilizada para o desenvolvimento do ambiente virtual proposto.

5.1. Ferramentas e Tecnologias Utilizadas

As ferramentas utilizadas serão apresentadas conforme sua função no projeto e serão divididas da seguinte forma:

- Ferramentas de desenvolvimento dos sistemas Robóticos
 - Framework ROS
 - Pacotes utilizados - MoveIt, Rviz e Gazebo
- Ferramentas para virtualização do sistema e disponibilização na web
 - Docker
 - VSCode
 - Github Codespaces
 - noVNC
 - Ambiente Gráfico XFCE4

5.1.1. Ferramentas de desenvolvimento dos sistemas Robóticos

A. ROS - *Robot Operating System*

A robótica contemporânea é um campo em constante evolução, impulsionado por avanços significativos em hardware e software. Para lidar com a crescente complexidade dos sistemas robóticos, surgiu a necessidade de plataformas de desenvolvimento que facilitem a criação, teste e implantação de aplicações. Nesse contexto, o *Robot Operating System* (ROS) estabeleceu-se como uma ferramenta fundamental, atuando como um middleware de código aberto que oferece um conjunto robusto de bibliotecas e ferramentas para o desenvolvimento de software robótico (OPEN SOURCE ROBOTICS FOUNDATION, 2014). Embora seu nome sugira um sistema operacional, o ROS é, na verdade, uma estrutura flexível que abstrai a complexidade do hardware, permitindo que pesquisadores e desenvolvedores se concentrem na lógica de controle e nas funcionalidades do robô.

Apresentaremos aqui o ROS de forma clara e acessível, abordando seus conceitos fundamentais, sua evolução do ROS 1 para o ROS 2, suas principais características, exemplos de aplicação e integração com outras ferramentas.

a) *Definição e propósito*

O *Robot Operating System* (ROS) é um conjunto de bibliotecas de software e ferramentas que auxiliam na construção de aplicações robóticas. Seu propósito principal é fornecer uma estrutura padronizada e modular para o desenvolvimento de software, promovendo a reutilização de código e a colaboração entre diferentes equipes e projetos (ROS, 2025b).

O papel do ROS no desenvolvimento de sistemas robóticos modernos é multifacetado. Ele atua como uma camada de abstração entre o hardware do robô e o software de aplicação, permitindo que os desenvolvedores escrevam código que seja independente das especificidades de um robô particular. Além disso, o ROS oferece ferramentas para simulação, visualização, depuração e gerenciamento de pacotes, acelerando significativamente o ciclo de desenvolvimento de robôs complexos.

b) *Arquitetura*

A arquitetura do ROS é baseada em um modelo distribuído e modular, em que diferentes funcionalidades são encapsuladas em componentes independentes que se comunicam entre si (Figura 1). Os principais componentes do ROS incluem:

- **Nós (Nodes):** Processos executáveis que realizam computações específicas (ROS WIKI, 2025).
- **Tópicos (Topics):** Canais de comunicação assíncrona que utilizam o modelo publish/subscribe.
- **Mensagens (Messages):** Estruturas de dados que encapsulam as informações trafegadas entre os nós.

- **Serviços (Services):** Comunicação síncrona do tipo requisição/resposta entre os nós. Utilizada para operações pontuais e únicas.
- **Ações (Actions):** Comunicação assíncrona que permite que o cliente possa executar outras tarefas enquanto a ação está em andamento. Utilizado para tarefas longas, permite o cancelamento e envia feedback durante sua execução.
- **Parâmetros (Parameter Server):** Configurações globais acessíveis pelos nós.
- **Bags:** Arquivos de log que armazenam dados das mensagens do ROS.
- **Master (ROS 1):** Componente central no ROS 1, responsável pelo registro e descoberta de nós, substituído no ROS 2 por uma arquitetura descentralizada baseada em DDS.

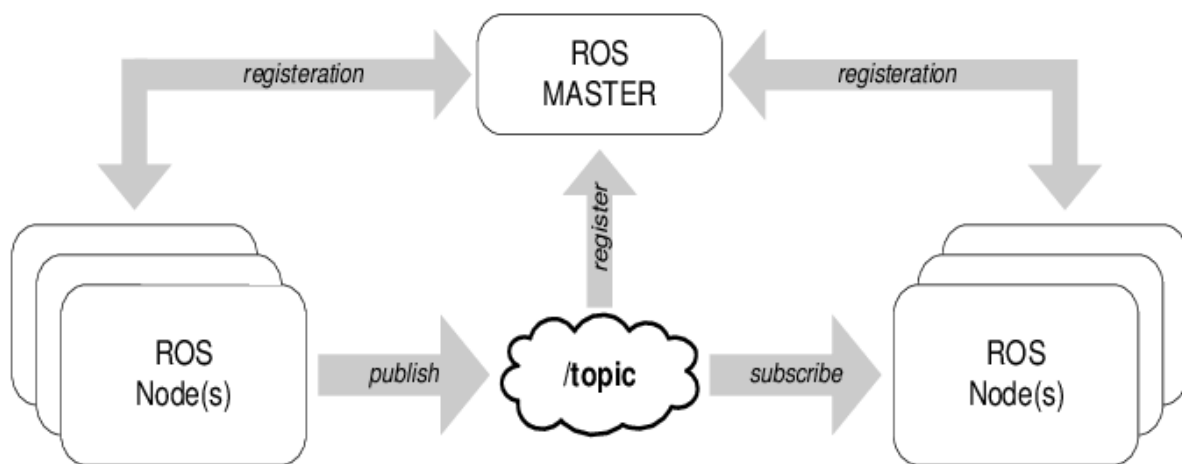


Figura 1 - Estrutura típica do framework ROS (ACHMAD et al., 2017)

c) Exemplo de Aplicação no ROS 1

A Figura 2 apresenta um exemplo simplificado de um sistema robótico implementado utilizando o framework ROS 1. Este exemplo ilustra um robô móvel equipado com uma câmera capaz de seguir um objeto específico — uma bola, por exemplo.

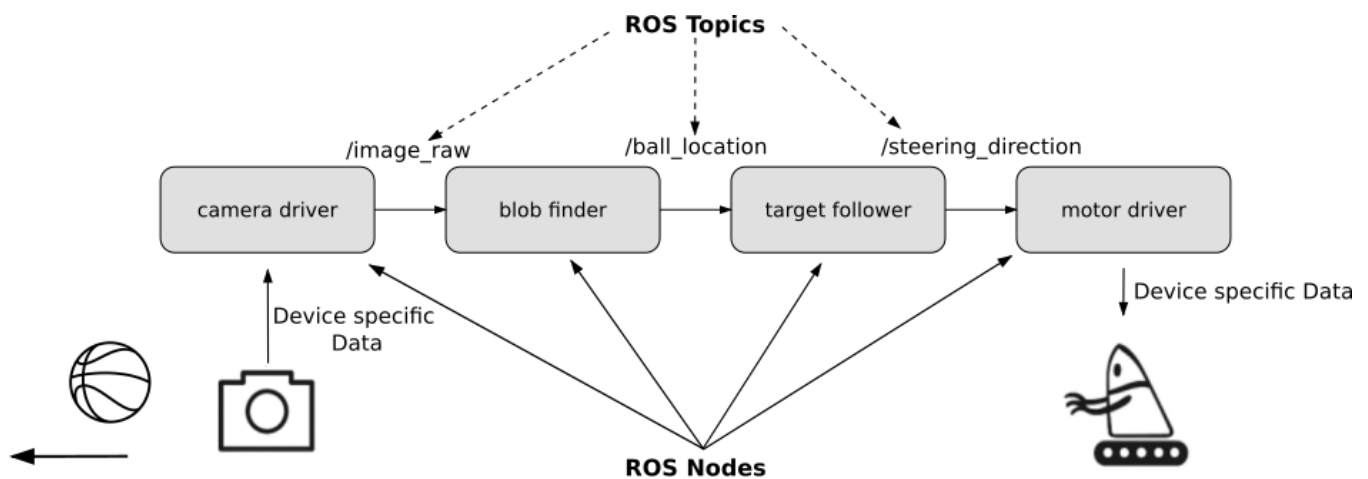


Figura 2 - Diagrama de blocos simplificado que demonstra a arquitetura modular de um sistema robótico construído com ROS. Diferentes funcionalidades (como driver de câmera, localizador de objetos, seguidor de alvo e driver de motor) são implementadas como nós independentes que se comunicam para realizar uma tarefa complexa(CHAUDHARY, 2021)

Neste sistema, cada funcionalidade é implementada como um **nó ROS independente**, promovendo a modularidade e a escalabilidade do sistema. A comunicação entre os nós ocorre por meio de **tópicos ROS**, utilizando o modelo de publicação e assinatura (*publish/subscribe*). As informações trafegam no formato de **mensagens ROS**, estruturadas de acordo com tipos de dados predefinidos.

A arquitetura é composta pelos seguintes nós:

- **Camera Driver:** Captura imagens da câmera e publica no tópico */image_raw*.
- **Blob Finder:** Processa a imagem, identifica a posição da bola e publica a informação no tópico */ball_location*.
- **Target Follower:** Recebe a localização da bola e calcula o comando de direção necessário, publicando no tópico */steering_direction*.
- **Motor Driver:** Recebe os comandos de direção e os converte em comandos específicos para os motores do robô.

Essa arquitetura permite que cada nó seja desenvolvido, testado e executado de forma independente, simplificando o desenvolvimento e facilitando a manutenção. Além disso, é possível substituir ou atualizar qualquer componente (por exemplo, trocar o algoritmo de detecção de objetos) sem alterar os demais nós, desde que a interface (mensagens e tópicos) permaneça a mesma.

d) Limitações do ROS 1

Apesar de suas inúmeras contribuições, o ROS 1 apresentava algumas limitações que se tornaram mais evidentes com o avanço da robótica e a demanda por sistemas mais robustos e escaláveis:

- **Comunicação Centralizada (ROS Master):** A dependência do ROS Master para o registro e busca de nós introduzia um ponto único de falha. Se o Master falhasse, a comunicação entre os nós seria interrompida, comprometendo a robustez do sistema (ROS, 2025a).
- **Ausência de Qualidade de Serviço (QoS):** O ROS 1 não possuía mecanismos nativos para garantir a Qualidade de Serviço (QoS), como priorização de mensagens, confiabilidade ou garantias de tempo real. Isso dificultava o desenvolvimento de aplicações críticas que exigiam comunicação determinística, como sistemas de controle de robôs industriais ou veículos autônomos (DYNAMICS BOSTON, 2025)(DYNAMICS BOSTON, 2025).
- **Segurança Limitada:** A segurança não era uma preocupação central no design do ROS 1. A comunicação entre os nós não era criptografada ou autenticada por padrão, tornando os sistemas vulneráveis a ataques e manipulações em ambientes não controlados (DYNAMICS BOSTON, 2025).
- **Suporte a Múltiplos Sistemas Operacionais:** Embora o ROS 1 fosse predominantemente utilizado em Linux, o suporte para outros sistemas operacionais, como Windows e macOS, era limitado ou experimental, dificultando o desenvolvimento em ambientes heterogêneos (DYNAMICS BOSTON, 2025).
- **Escalabilidade e Sistemas Embarcados:** O ROS 1 não foi projetado para lidar eficientemente com grandes frotas de robôs ou para ser executado em sistemas embarcados com recursos computacionais limitados. Sua arquitetura e sobrecarga de comunicação não eram ideais para essas aplicações (DYNAMICS BOSTON, 2025).

Essas limitações, especialmente a falta de garantias de tempo real e segurança, motivaram o desenvolvimento de uma nova geração da plataforma: o ROS 2.

e) ROS 2: Motivação e Melhorias

O ROS 2 foi concebido para superar as deficiências do ROS 1 e atender às crescentes demandas da robótica moderna, especialmente em aplicações industriais e de missão crítica. A motivação para sua criação centrou-se na necessidade de uma plataforma mais robusta, segura, escalável e com suporte a tempo real (DYNAMICS BOSTON, 2025)

f) Principais Melhorias em Relação ao ROS 1

As melhorias introduzidas no ROS 2 são significativas e abordam diretamente as limitações de seu predecessor:

- **Suporte a Sistemas Embarcados e Tempo Real:** Uma das maiores inovações do ROS 2 é a sua capacidade de operar em sistemas embarcados e oferecer comunicação com garantias de tempo real. Isso é crucial para aplicações que exigem respostas rápidas e previsíveis, como controle de robôs colaborativos, cirurgias robóticas e veículos autônomos. O ROS 2 alcança isso através da integração com o *Data Distribution Service* (DDS)(RTI COMMUNITY, 2025).
- **Data Distribution Service (DDS):** O DDS é um padrão de comunicação middleware para sistemas distribuídos em tempo real. No ROS 2, o DDS substitui o ROS Master e o sistema de comunicação customizado do ROS 1. O DDS oferece uma arquitetura descentralizada, onde os nós se descobrem e se comunicam diretamente, eliminando o ponto único de falha. Além disso, o DDS fornece mecanismos de Qualidade de Serviço (QoS) configuráveis, permitindo que os desenvolvedores especifiquem requisitos de confiabilidade, latência e durabilidade para a comunicação (RTI COMMUNITY, 2025).
- **Segurança:** A segurança foi uma prioridade no design do ROS 2. Através do DDS, o ROS 2 incorpora recursos de segurança como autenticação, criptografia e controle de acesso, protegendo a comunicação entre os nós contra acessos não autorizados e manipulações. Isso é fundamental para robôs que operam em ambientes públicos ou industriais, onde a segurança cibernética é essencial (RTI COMMUNITY, 2025).
- **Escalabilidade:** A arquitetura descentralizada do DDS e aprimoramentos internos permitem que o ROS 2 suporte um número maior de nós e uma maior taxa de transferência de dados, tornando-o mais adequado para aplicações com grandes frotas de robôs ou sistemas robóticos complexos (DYNAMICS BOSTON, 2025).
- **Suporte a Múltiplos Sistemas Operacionais:** O ROS 2 oferece suporte nativo e completo para Linux, Windows e macOS, além de plataformas embarcadas através do micro-ROS. Essa compatibilidade multiplataforma facilita o desenvolvimento e a implantação de aplicações em diversos ambientes (ROS, 2025a)
- **Múltiplas Linguagens de Programação:** O ROS 2 expandiu o suporte a linguagens de programação, incluindo C++, Python, Java, C# e MATLAB, oferecendo maior flexibilidade para os desenvolvedores (RICARDO TELLEZ, 2019).

A figura a seguir extraída de (PENG *et al.*, 2023) compara as arquiteturas do ROS 1 e 2.

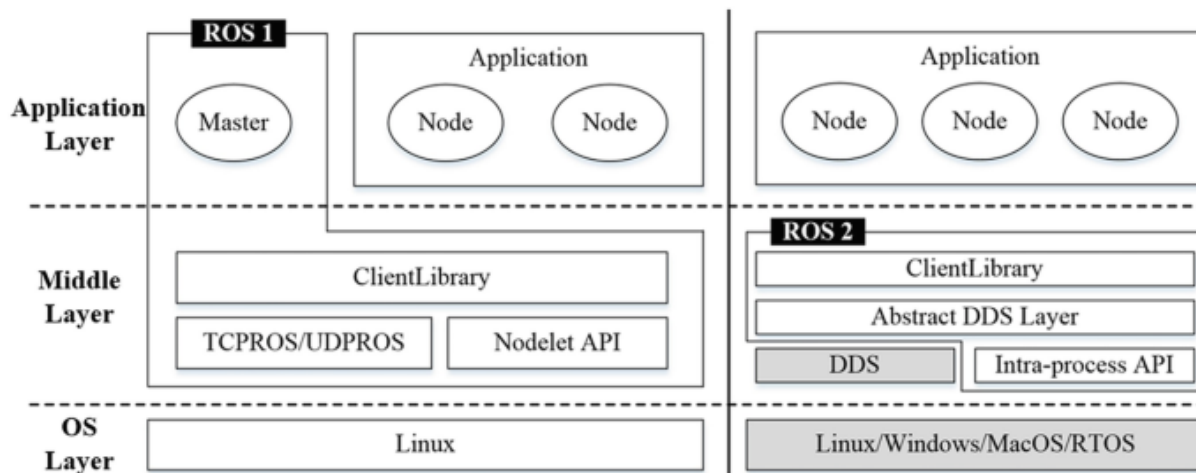


Figura 3 - Comparação da arquitetura ROS1 e ROS2(PENG et al., 2023)

Embora a versão 2 tenha avanços consideráveis a versão 1 está mais consolidada, principalmente no meio acadêmico, e atualmente possui maior quantidade de recursos disponíveis. Por conta disso, neste projeto optou-se pela utilização predominante da versão 1 na última distribuição LTS (*Long Term Support*) do ROS a Noetic Ninjemys cujo suporte termina em 2025, mas pela disponibilização de um ambiente e alguns exemplos em ROS 2, na versão LTS Jazzy Jalisco. Acreditamos que a plataforma possa ser utilizada para auxílio na transição entre os sistemas através da adição de tutoriais e material comparativo entre eles.

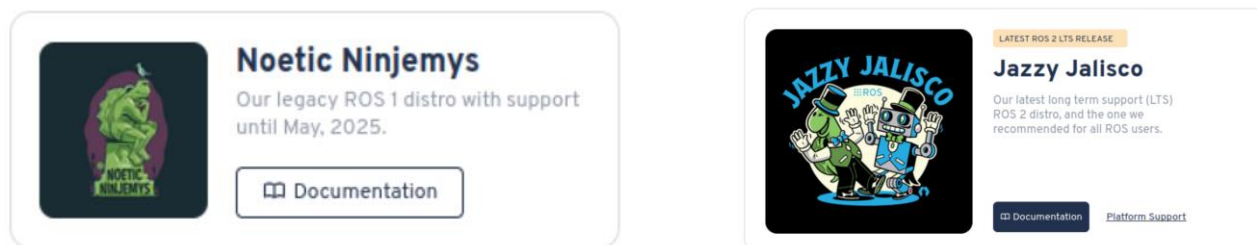


Figura 4 - Versões do ROS utilizadas na plataforma: ROS 1 na versão Noetic Ninjemys e ROS2 na versão Jazzy Jalisco (ROS - WIKI, 2023)

B. Gazebo

O Gazebo é parte do Player Project (PLAYER, 2014) e permite a simulação de aplicações robóticas e de sensores em ambientes tridimensionais internos e externos. Ele possui uma arquitetura Cliente/Servidor e um modelo de comunicação entre processos baseado em tópicos de Publicação/Assinatura.

O Gazebo possui uma interface padrão do Player e, adicionalmente, uma interface nativa. Os clientes do Gazebo podem acessar seus dados através de uma memória compartilhada. Cada objeto de simulação no Gazebo pode ser associado a um ou mais controladores que processam comandos para controlar o objeto e geram o estado desse objeto. Os dados gerados pelos controladores são publicados na memória compartilhada usando interfaces do Gazebo (Ifaces). As Ifaces de outros processos podem ler os dados da memória compartilhada, permitindo assim a comunicação entre processos entre o software de controle do robô e o Gazebo, independentemente da linguagem de programação ou da plataforma de hardware do computador.

No processo de simulação dinâmica, o Gazebo pode acessar motores físicos de alto desempenho como *Open Dynamics Engine* (ODE)(KRISTENSEN *et al.*, 2019; LEE *et al.*, 2018; PYBULLET, 2022; SHERMAN; EASTMAN, 2024) que são usados para simulação física de corpos rígidos. O *Object-Oriented Graphics Rendering Engine* (SOFTWARE, 2016) fornece a renderização gráfica 3D dos ambientes do Gazebo.

O Cliente envia dados de controle e coordenadas dos objetos simulados para o Servidor, que realiza o controle em tempo real do robô simulado. É possível realizar uma simulação distribuída colocando o Cliente e o Servidor em diferentes máquinas. Implementar o Plugin ROS para o Gazebo ajuda a criar uma interface de comunicação direta com o ROS, permitindo assim controlar os robôs simulados e reais usando o mesmo software. Isso fornece uma ferramenta de simulação eficaz para testes e desenvolvimento de sistemas robóticos reais.

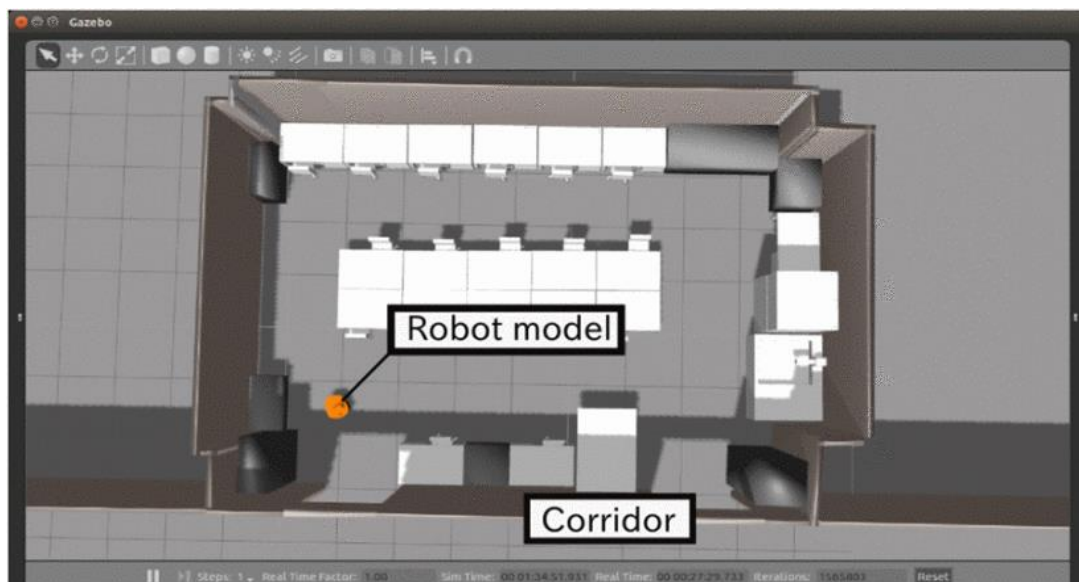


Figura 5 - ROS Gazebo utilizado na simulação de robôs (TAKAYA *et al.*, 2016)

O Gazebo é um dos simuladores mais utilizados no ROS (MARIAN *et al.*, 2020) o que faz com que vários fabricantes de robôs ofereçam pacotes ROS específicos para utilização no Gazebo. Isso faz do conjunto ROS + Gazebo uma escolha bastante adequada para a utilização em plataformas para o ensino de robótica sem nenhum custo.

a) Integração Gazebo + ROS

A integração entre ROS e Gazebo é fundamental para simulações robóticas eficazes. O Gazebo oferece um ambiente de simulação física precisa, onde modelos de robôs e seus arredores podem interagir de forma realista. O ROS, por sua vez, fornece a infraestrutura de comunicação e as ferramentas de software que permitem que os componentes do robô (sensores, atuadores, controladores) operem dentro desse ambiente simulado. Essa integração é alcançada principalmente através de pacotes ROS específicos, como o *gazebo_ros_pkgs*, que fornecem as interfaces necessárias para a troca de mensagens, serviços e reconfiguração dinâmica entre o ROS e o Gazebo (ROS - WIKI, 2023).

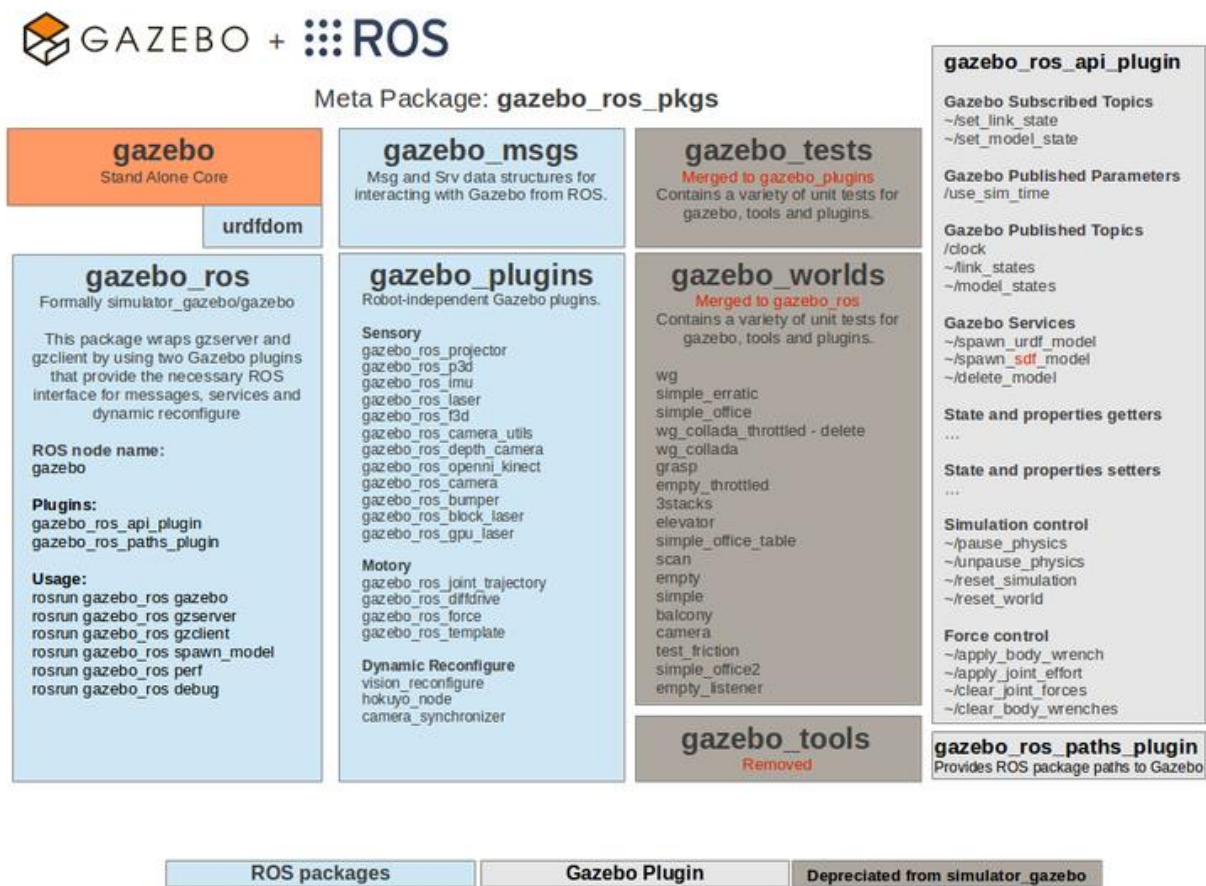


Figura 6 - Diagrama da integração entre o simulador Gazebo e o middleware ROS utilizando o meta-pacote *gazebo_ros_pkgs*(GAZEBO, 2025)

A integração é facilitada por diversos componentes e conceitos:

- **gazebo_ros_pkgs:** Este metapacote contém wrappers e bibliotecas que permitem a comunicação entre o ROS e o Gazebo. Ele inclui plugins que podem ser carregados no Gazebo para expor funcionalidades do simulador ao ROS, como dados de sensores e controle de atuadores. Exemplos plugins de atuadores: gazebo_ros_diff_drive que permite o controle de robôs com diferencial (tipo TurtleBot), gazebo ros_joint trajectory, permite controle por trajetória no MoveIt e de sensores: gazebo_ros_laser, publica dados de laser/LiDAR, gazebo_ros_imu, publica dados de acelerômetros (IMU).
- **URDF (Unified Robot Description Format):** O URDF é um formato XML para descrever modelos de robôs no ROS. O Gazebo pode interpretar arquivos URDF, permitindo que os mesmos modelos de robôs usados no ROS sejam simulados no Gazebo. Isso garante consistência entre o ambiente real e o simulado [2].
- **Plugins Gazebo:** Os plugins são bibliotecas de código que podem ser carregadas no Gazebo para estender sua funcionalidade. Existem plugins ROS específicos que permitem a publicação de dados de sensores (câmeras, LiDARs, IMUs) para tópicos ROS e a subscrição a comandos de controle (velocidade, posição conjunta) de tópicos ROS. Isso permite que os algoritmos ROS interajam diretamente com o robô simulado.
- **ros_control :** Este framework ROS fornece uma maneira padronizada de controlar robôs. Ele pode ser integrado ao Gazebo através de plugins, permitindo que os controladores ROS gerenciem os atuadores do robô simulado com alta fidelidade.

b) Fluxo de integração

A integração segue o fluxo descrito abaixo:

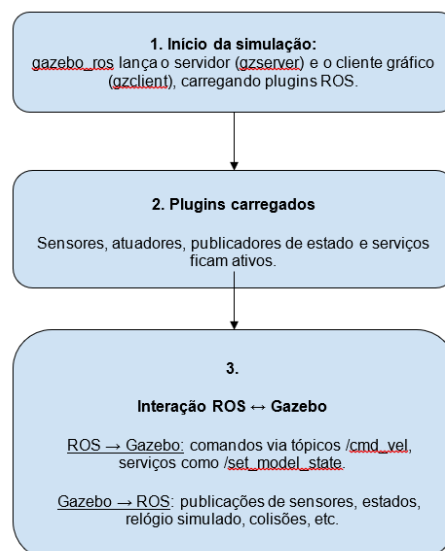


Figura 7 - Fluxo da interação entre Gazebo e ROS

C. MoveIt!

O MoveIt! é uma plataforma de software de código aberto para planejamento de movimento de robôs, manipulação e cinemática, construída sobre o Robot Operating System (ROS). Conforme destacado por (SUCAN; CHITTA, 2018), o MoveIt! integra diversos componentes cruciais para o planejamento de movimento, como a cinemática direta e inversa, a geração de trajetórias, a evasão de colisões e a percepção ambiental (CHITTA; SUCAN; COUSINS, 2012). Esta integração facilita a criação de soluções completas de robótica, reduzindo significativamente o tempo e o esforço necessários para desenvolver algoritmos personalizados.

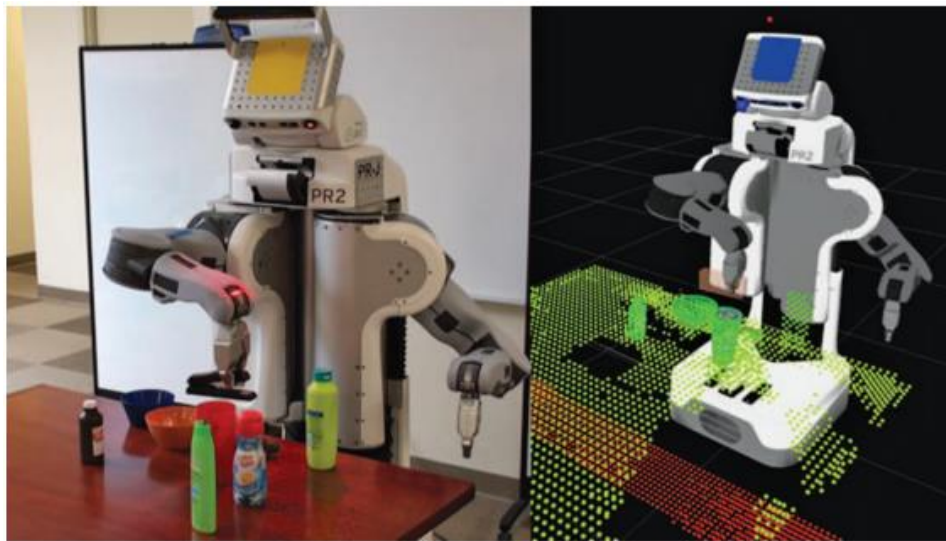


Figura 8 - -Uma imagem do robô PR2 usando as capacidades de planejamento de movimento no ROS para executar um plano de movimento restrito como parte de uma tarefa de pegar e colocar, mantendo o objeto (um grampeador) na posição horizontal. A visualização à direita mostra a representação do ambiente, construída a partir de sensores embarcados, usada pelo robô para o planejamento de movimento (CHITTA; SUCAN; COUSINS, 2012)

Seu principal propósito é fornecer um conjunto abrangente de ferramentas para que robôs possam planejar e executar movimentos complexos em ambientes dinâmicos e com obstáculos, tornando-o uma ferramenta essencial na robótica moderna por simplificar o desenvolvimento de aplicações robóticas que exigem navegação e manipulação precisas.

a) Funcionalidades do MoveIt!

O MoveIt! oferece diversas funcionalidades que o tornam uma solução robusta para o planejamento de movimento:

- **Planejamento de Movimento:** Gera trajetórias de alta liberdade em ambientes com obstáculos, evitando colisões e otimizando o caminho do robô. A ferramenta suporta múltiplos algoritmos de

planejamento, como o RRT (Rapidly-exploring Random Trees) (LI; CHEN, 2022) e o PRM (Probabilistic Roadmap) (SHORT *et al.*, 2016), oferecendo aos desenvolvedores uma vasta gama de opções para otimizar o desempenho de seus sistemas robóticos.

- **Manipulação:** Permite a análise e interação com o ambiente, incluindo a geração de garras para manipulação de objetos.
- **Cinemática Inversa:** Resolve as posições das juntas de um robô para alcançar uma pose específica, mesmo em braços com muitos graus de liberdade.
- **Controle:** Executa trajetórias de juntas parametrizadas no tempo para controladores de hardware de baixo nível através de interfaces comuns.
- **Percepção 3D:** Conecta-se a sensores de profundidade e nuvens de pontos, utilizando estruturas como Octomaps para representar o ambiente.
- **Verificação de Colisão:** Evita obstáculos usando primitivas geométricas, malhas ou dados de nuvem de pontos.

b) Integração com Gazebo e RViz

Um dos pontos fortes do MoveIt! é a sua integração com o ROS, que permite uma comunicação eficiente entre diferentes módulos do sistema robótico (COUSINS, 2010). Esta integração facilita a simulação e a implementação real, pois os desenvolvedores podem testar e ajustar seus algoritmos em ambientes simulados antes de aplicá-los em robôs físicos, minimizando riscos e custos.

O MoveIt! se integra perfeitamente com outras ferramentas do ecossistema ROS, como o Gazebo e o RViz, para fornecer um ambiente completo de simulação e visualização:

- **Gazebo:** Um simulador de robôs 3D que permite testar e validar algoritmos de planejamento de movimento em um ambiente virtual realista antes de implantá-los em hardware real.
- **RViz:** Uma ferramenta de visualização 3D que permite aos usuários visualizar o modelo do robô, o ambiente, as trajetórias planejadas e os resultados da detecção de colisões em tempo real (ROS, 2025b).

c) Funcionamento do MoveIt! - Move_group

O nó *move_group* é o componente central do MoveIt!, atuando como uma interface unificada para todas as funcionalidades de planejamento e execução de movimento. Ele orquestra a comunicação entre os diversos módulos do MoveIt! e o restante do ecossistema ROS, conforme ilustrado na Figura 1. Sua arquitetura modular permite que desenvolvedores e pesquisadores interajam

com o MoveIt! de diversas formas, seja através de interfaces de usuário, como o plugin RViz, ou programaticamente via APIs em C++ e Python.

Uma das principais responsabilidades do *move_group* é gerenciar as requisições de planejamento de movimento. Quando uma aplicação solicita que o robô se mova para uma determinada pose ou execute uma tarefa de manipulação, como um pick (pegar) ou place (colocar), essa requisição é enviada ao *move_group*. Ele então coordena o processo de encontrar uma trajetória válida que leve o robô ao seu destino, evitando colisões com o ambiente e com o próprio robô.

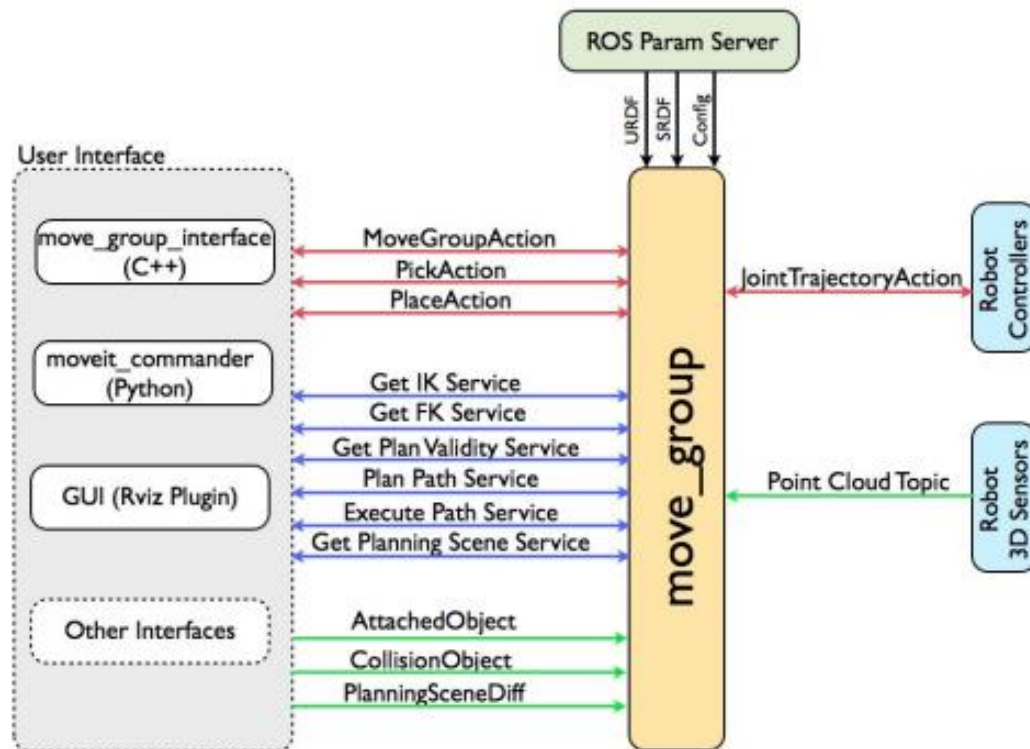


Figura 9 - Diagrama da arquitetura do nó *move_group* do MoveIt!, ilustrando suas interações com a interface do usuário, servidor de parâmetros ROS, controladores do Robô e sensores 3D(SUCAN; CHITTA, 2018)

Para realizar o planejamento, o *move_group* interage com o servidor de parâmetros do ROS, onde estão configurados os parâmetros do robô, como seus limites de junta e modelos cinemáticos. Ele também se comunica com os sensores 3D do robô, que fornecem dados sobre o ambiente. Essas informações são cruciais para a construção de um mapa de obstáculos e para a detecção de colisões em tempo real, garantindo a segurança da operação do robô.

Após o planejamento bem-sucedido de uma trajetória, o *move_group* é responsável por enviá-la aos controladores de baixo nível do robô. Isso é feito através de ações de trajetória de junta, que traduzem o plano de movimento abstrato em comandos específicos que o hardware do robô pode

executar. Essa separação de responsabilidades entre planejamento e controle permite que o MoveIt! seja compatível com uma ampla variedade de plataformas robóticas.

Além das funcionalidades de planejamento e execução, o *move_group* também oferece serviços para cinemática inversa e direta, permitindo que os usuários consultem as posições das juntas para uma dada pose do efetuador final, ou vice-versa. Ele também gerencia objetos de colisão e a cena de planejamento, permitindo que o ambiente do robô seja dinamicamente atualizado e considerado durante o planejamento de movimento.

O *move_group* é altamente configurável, o que significa que seus algoritmos de planejamento, modelos de robô e parâmetros de ambiente podem ser ajustados para atender às necessidades específicas de cada aplicação. Essa flexibilidade é fundamental para adaptar o MoveIt! a diferentes cenários, desde robôs industriais em ambientes estruturados até robôs de pesquisa em ambientes não estruturados e dinâmicos.

Por todas estas características o MoveIt! é amplamente utilizada em diversas aplicações industriais (BURGH-OLIVÁN; ARAGÜÉS; LÓPEZ-NICOLÁS, 2023; KRISTENSEN *et al.*, 2019; MEGALINGAM *et al.*, 2021) e na academia. Além disso, o MoveIt suporta múltiplos algoritmos de planejamento, como o RRT e o PRM, oferecendo aos desenvolvedores uma vasta gama de opções para otimizar o desempenho de seus sistemas robóticos.

5.1.2 Ferramentas para virtualização do sistema e disponibilização na web

A seguir serão apresentadas as ferramentas utilizadas para virtualização dos sistemas robóticos e codificação da plataforma.

A. Docker e virtualização de ambientes

O Docker é uma plataforma de código aberto que permite criar, empacotar e executar aplicações em ambientes isolados chamados de containers. Essa tecnologia surgiu como uma solução para o problema clássico de compatibilidade entre ambientes de desenvolvimento e produção, promovendo portabilidade, reprodutibilidade e escalabilidade dos sistemas (DOCKER, 2025).

Ao contrário das máquinas virtuais, que virtualizam todo um sistema operacional, os containers compartilham o kernel do sistema operacional do host (máquina física ou servidor) e executam apenas os componentes necessários para a aplicação, tornando-os mais leves, rápidos e eficientes em termos de recursos.

O funcionamento do Docker baseia-se em alguns conceitos fundamentais:

- **Imagem (Image):** é um pacote imutável que contém tudo o que é necessário para rodar uma aplicação — sistema operacional mínimo, bibliotecas, dependências e o próprio código. As imagens funcionam como modelos para a criação dos containers.
- **Container:** é uma instância em execução de uma imagem. Ele é isolado do sistema host e de outros containers, garantindo que a aplicação execute sempre nas mesmas condições, independentemente do ambiente.
- **Dockerfile:** é um arquivo de texto que define, de forma declarativa, as instruções necessárias para construir uma imagem. Cada linha no Dockerfile representa um comando (como instalar pacotes, copiar arquivos ou definir comandos de inicialização).
- **Build:** é o processo de construção de uma imagem Docker a partir de um Dockerfile. O comando `docker build` lê esse arquivo e gera uma imagem pronta para ser executada.
- **Run:** é o comando utilizado para criar e executar um container a partir de uma imagem. Por exemplo, `docker run` inicializa uma aplicação isolada conforme definido na imagem.

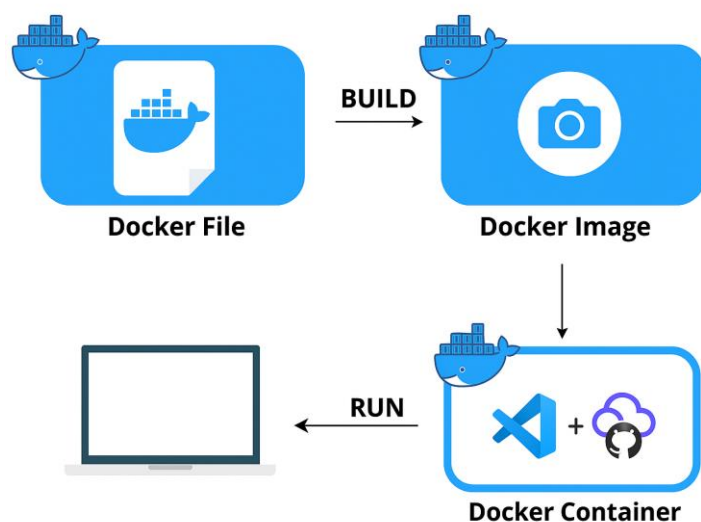


Figura 10 - Imagem ilustrativa do funcionamento do Docker A partir das definições de um dockerfile uma imagem é montada (build) e pode ser executada em diversas instâncias isoladas (Docker Container) podendo ser executadas tanto localmente quanto em nuvem com uso de ferramentas como vscode e o Codespaces

Além disso, o Docker opera em um modelo cliente-servidor. O Docker Host é a máquina onde os containers são executados, e o Docker Daemon é o serviço que gerencia imagens, containers, redes e volumes. O usuário interage com o daemon através do Docker Client, geralmente via linha de comando, que envia comandos como `build`, `run`, `stop` e `rm`.

Ferramentas como o VSCode (VISUAL STUDIO CODE, 2025) e o GitHub Codespaces (GITHUB, 2025) integram-se diretamente com Docker, proporcionando ambientes de desenvolvimento portáteis e acessíveis. O VSCode, por meio de extensões como Dev Containers, permite abrir um ambiente de desenvolvimento completo dentro de um container Docker, garantindo que todo o setup de bibliotecas, dependências e versões seja replicável. Já o GitHub Codespaces leva essa ideia para a nuvem, oferecendo ambientes de desenvolvimento completos e baseados em containers diretamente no navegador, integrados ao repositório GitHub. Isso elimina a necessidade de configurações locais e garante que qualquer usuário, em qualquer máquina, acesse um ambiente de desenvolvimento idêntico, com apenas um clique.

Essa abordagem fortalece práticas de ciência aberta, reprodutibilidade e colaboração, especialmente em projetos acadêmicos e de pesquisa.

Uma contribuição importante deste projeto foi a codificação de diferentes dockerfiles contendo toda a especificação dos ambientes de trabalho em ROS. Estes dockerfiles, juntamente com o arquivo devcontainer.json, responsável pela execução do container a partir do vscode, são os grandes responsáveis pela virtualização e abstração de todas as configurações necessárias para criação do ambiente para o usuário. Dentre as informações que compõe os dockerfiles deste projeto estão:

- Versão do ROS (noetic ou jazzy) a partir de uma distribuição oficial (OSRF), versão base (minima) ou desktop-full (completa)
- Pacotes necessários para execução daquela atividade e suas dependências - gazebo, gazebo controllers, moveit e outros
- Configuração do noVNC que permite criar e acessar um ambiente gráfico de um computador remoto diretamente pelo navegador web.
- Configuração do ambiente gráfico remoto XFCE4 adotado neste projeto por conta de sua leveza e funcionalidade
- Scripts variados de inicialização do workspace ROS de trabalho dentro do ambiente virtual, compilação de exemplos, abstração de procedimentos do ROS

B. noVNC e a criação de ambientes virtuais

O noVNC (NOVNC, 2025) é uma ferramenta que permite acessar um ambiente gráfico de um computador remoto diretamente pelo navegador web, sem precisar instalar nenhum software adicional. Ele funciona como uma interface web para o protocolo VNC (Virtual Network Computing), que é tradicionalmente usado para acesso remoto a desktops.

Quando usamos containers Docker, normalmente interagimos com eles via terminal (linha de comando). Porém, alguns softwares — como simuladores (Gazebo, RViz) ou ambientes de desenvolvimento gráficos (VSCode, JupyterLab, interfaces Linux com XFCE, por exemplo) — exigem uma interface gráfica para funcionar plenamente.

O noVNC permite criar um ambiente gráfico dentro do container Docker, que pode ser acessado via navegador como se fosse um desktop remoto. Isso torna possível rodar aplicações gráficas, ambientes Linux completos e ferramentas de simulação ou desenvolvimento, tudo dentro de um container Docker e acessível de qualquer computador conectado à internet.

a. Criação de ambientes remotos com o noVNC e o Docker

A criação dos ambientes virtuais a partir dos docker containers com o noVNC ocorrem da seguinte forma:

1. O container roda um servidor gráfico (como XFCE) e um servidor VNC.
2. O noVNC converte a interface VNC em uma interface web acessível pelo navegador.
3. Ao acessar um link (neste caso, <http://localhost:6080> ou um endereço remoto), o usuário visualiza e interage com o ambiente gráfico do container.

Isso possibilita:

- Executar o **Gazebo**, **RViz** ou qualquer ferramenta gráfica de simulação robótica dentro de um container.
- Usar um ambiente Linux completo, com terminal gráfico, navegador, editor de texto, **VSCode**, diretamente no browser.
- Facilitar o acesso de estudantes e desenvolvedores, sem necessidade de instalar sistemas operacionais ou softwares locais.

C. Interface do ambiente virtual - XFCE4

O XFCE4 (XFCE, 2025) fornece o ambiente gráfico dentro dos containers Docker utilizados neste projeto. Ele permite que usuários acessem, via navegador e com suporte do noVNC, um desktop Linux completo, onde podem executar ferramentas gráficas necessárias para o desenvolvimento e simulação em ROS, de forma portátil, leve e sem dependências de instalação no computador local.

Neste projeto o XFCE4 foi escolhido por permitir a execução de aplicações como o VSCode, o RViz, Gazebo, navegadores e outras que exigem um ambiente gráfico dentro do container em uma interface de desktop leve, eficiente e funcional

O XFCE4 é instalado e configurado dentro do container Docker, funcionando como a interface gráfica do ambiente e é transmitido para o navegador do usuário através do noVNC que converte o ambiente gráfico em uma interface web acessível remotamente, sem necessidade de instalar um cliente VNC.

Desta forma foi possível criar um ambiente de desenvolvimento completo, incluindo simuladores e editores gráficos (VSCode) que pode ser acessado localmente via navegador ou na nuvem através do GitHub Codespaces ou servidores remotos.

As ferramentas básicas incluídas no ambiente virtual deste projeto foram:

- VSCode
- Framework ROS 1 ou 2
- RViz, rqt e outras ferramentas gráficas do ROS
- Simulador Gazebo
- Navegador Firefox

5.2 Metodologia de desenvolvimento do projeto

O projeto foi desenvolvido seguindo as etapas previstas no planejamento e descritas a seguir:

Etapla 01 – Estudos dos referenciais teóricos e ferramentas utilizados no projeto – ROS, Gazebo e Movelt

Permitiu conhecer os aspectos práticos da programação e simulação de sistemas robóticos utilizando o ROS 1, Gazebo e pacotes associados num ambiente containerizado (Docker + VSCode) localmente.

Etapla 02 - Análise sistemática de ambientes/plataformas virtuais existentes

Permitiu analisar alguns dos principais e mais recentes ambientes virtuais existentes publicados na literatura, recursos e conhecimentos computacionais necessários para sua implementação, nível de complexidade. A partir desta análise foi possível estabelecer os requisitos para o sistema aqui proposto, considerando-se as limitações de tempo e recursos do projeto.

Etapa 03 – Criação de exemplos para demonstração de conceitos básicos de robótica

Possibilitou o estabelecimento de parâmetros pedagógicos da plataforma virtual considerando o ponto de vista dos alunos, organização temática dos módulos, aulas e projetos e sua implementação/codificação a partir desta lógica. Nesta etapa foi realizada a documentação dos exemplos e atividades da plataforma

Etapa 04 - Desenvolvimento do ambiente virtual de ensino

Permitiu o estudo e codificação da execução da plataforma virtual a partir de ambientes em nuvem. Foram analisadas diferentes alternativas para virtualização dos ambientes virtuais como Binderhub, github classroom, servidores remotos, optando-se, por fim, pela utilização do Codespaces. A partir desta definição, foram selecionados os demais elementos da virtualização - noVNC, XFCE4, tendo sido escolhido o github Pages como serviço para disponibilização da plataforma gratuita.

Nesta etapa foram codificados os dockerfiles, devcontainer.json e scripts necessários a implementação do ambiente virtual, conforme os requisitos de cada atividade proposta. Nesta etapa também foi definida a concepção geral e a arquitetura como será apresentado em seguida.

5.2.1 Metodologia para desenvolvimento do ambiente virtual de ensino

O ambiente virtual de ensino foi desenvolvido ao longo das etapas 02, 03 e 04 do projeto tendo sido adotada a seguinte metodologia:

1. Definição dos requisitos
2. Definição da concepção da plataforma
3. Estabelecimento da arquitetura
4. Codificação dos ambientes virtuais
5. Codificação do material didático

a) Requisitos da plataforma

Os requisitos foram estabelecidos a partir da análise de trabalhos anteriormente publicados, das limitações de recursos e tempo deste projeto, bem como das contribuições propostas por este projeto. Neste sentido o trabalho de (KRUMINS *et al.*, 2024) foi utilizado como referência de formato.

Tabela 1 - Requisitos da plataforma para ensino de robótica proposta neste trabalho

Requisito	Justificativa
1. Tempo mínimo de configuração para alunos iniciantes: - não é necessário acesso de administrador no computador do aluno; - compatível com diferentes sistemas operacionais.	Configurar um novo sistema operacional, ROS e robôs físicos pode ser muito intimidador e apresenta alto risco de perda de dados quando feito por um não especialista. O tempo necessário para fazer o software funcionar tira o foco do engajamento cognitivo, e os instrutores são forçados a gastar recursos com suporte técnico. O aluno pode não saber se todo esse esforço vale a pena no início da jornada de aprendizado. Reduzir essa barreira de entrada, minimizando o esforço de configuração, permite que ferramentas profissionais de robótica (como Linux e ROS) se tornem um recurso educacional acessível.
2. Possibilidade de utilização dos códigos tanto no computador local, quanto no ambiente em nuvem	Embora interessante, a utilização de ambientes em nuvem tem limitações de tempo e custo (github Codespaces limita a 60hs/mês (geral) ou 80hs/mês (estudantes)) com limite no tamanho dos containers. A possibilidade de utilização local elimina essa limitação e possibilita o desenvolvimento pleno pelos alunos. O que interessante, principalmente, para o desenvolvimento de estudos e pesquisas na área.
3. Ambiente com exemplos e documentação em português e inglês	Ainda hoje há dificuldade dos estudantes em acessar documentos e sistemas em língua inglesa. O uso do português no material visa eliminar uma possível barreira ao aprendizado.
4. Disponibilidade de recursos de aprendizagem complementares como vídeos, acesso a sites educacionais gratuitos, entre outros	A plataforma deve poder ser utilizada pelo estudante sem a mediação direta de professores, podendo se acessada como recurso de aprendizagem em metodologias de ensino assíncronas
5. Possibilidade de agilizar o desenvolvimento por meio de simulações.	A oportunidade de usar simulações acelera frequentemente o desenvolvimento de software, especialmente quando o aluno continua na fase de estruturar o código e desenvolver os componentes básicos do algoritmo. O uso de robôs físicos é mais demorado e envolve riscos relacionados a problemas de hardware. Aprender o conceito de simulação, uma ferramenta amplamente adotada na robótica, permite que os alunos

	desenvolvam programas rapidamente e com baixo custo em um ambiente livre de riscos.
6. Aberto e personalizável.	O uso de soluções de código aberto na educação traz muitos benefícios, como maior possibilidade de personalização, ampla disponibilidade de ferramentas, menor risco de descontinuação do serviço, melhoria contínua por uma comunidade colaborativa e ausência de taxas de licença.

b) Concepção da plataforma

A partir dos requisitos e considerando-se as limitações de:

- A plataforma não seria hospedada em servidor próprio, devendo utilizar um serviço gratuito
- Não haveria disponibilidade de equipe de TI para manutenção da plataforma em operação
- A plataforma seria desenvolvida durante o período do projeto de pós-doutorado máximo de um ano por pesquisador com conhecimento na área de robótica, mas não na área de desenvolvimento de sistemas computacionais em nuvem

Chegou-se à concepção de plataforma ilustrada a seguir:

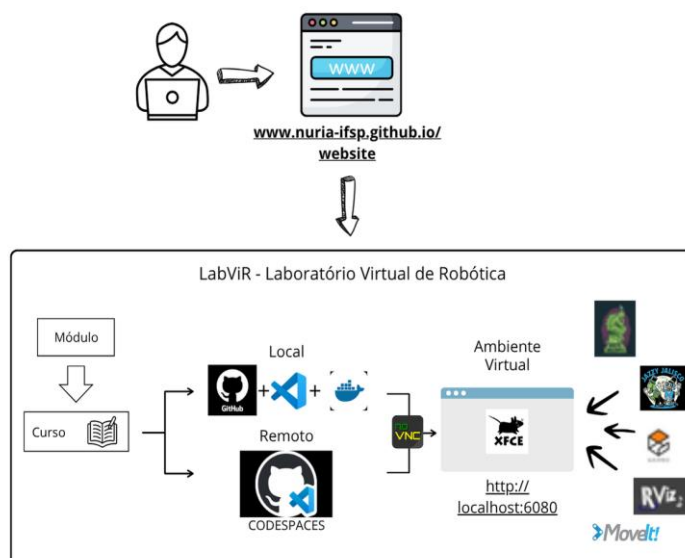


Figura 11 - Diagrama de concepção da plataforma LabViR - Laboratório Virtual de Robótica

Por esta concepção o aluno deveria acessar o site da plataforma na internet, em que teria acesso aos módulos e cursos. Ao escolher um curso teria a opção de baixar o repositório localmente a partir do github ou de executá-lo no github Codespaces (nos dois casos ele já deve ter conta gratuita no github). Ao executar a aplicação dentro do container (localmente ou em nuvem) ele inicia o servidor noVNC e pode acessar, em seu navegador o endereço do ambiente virtual. Neste ambiente ele terá a sua disposição todo o ambiente configurado para execução do ROS na versão escolhida, com as ferramentas necessárias para realização dos exercícios/atividades que são disponibilizados no pacote através de arquivos em formato markdown ([README.md](#)).

c) Arquitetura da Plataforma

A arquitetura da plataforma foi desenhada para ser modular, escalável e compatível com diferentes ambientes operacionais. Ela se organiza em quatro camadas principais ilustradas na Figura 12 e descritas em seguida.

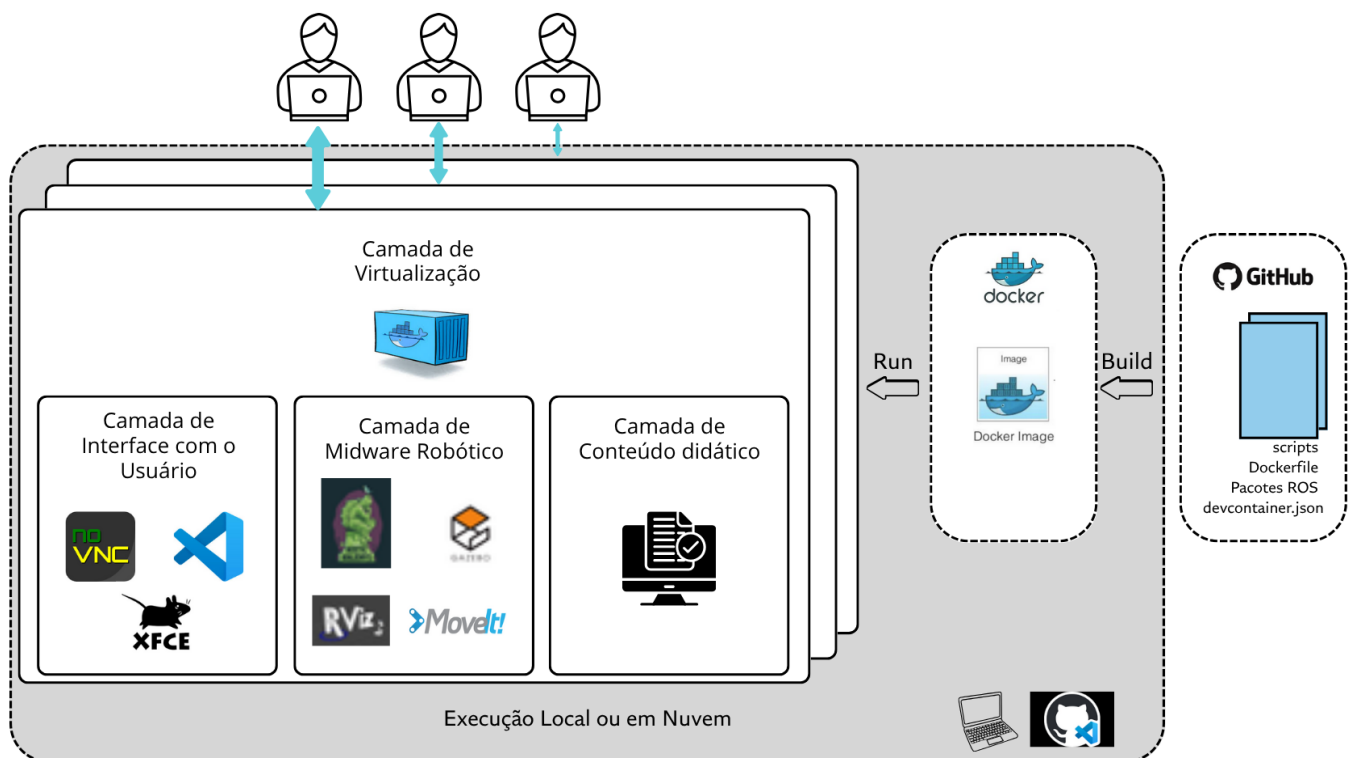


Figura 12 - Arquitetura da Plataforma LabViR. Os usuários tem acesso aos repositórios no Github que contém os arquivos de configuração utilizados para criação da imagem e execução dos containers de maneira isolada. Cada container (Camada de Virtualização) contém a interface com o usuário, os arquivos de configuração (middleware robótico) necessários para execução das atividades (conteúdo didático). Os containers podem ser executados localmente ou em nuvem.

- Camada de Virtualização

Responsável por garantir a portabilidade, isolamento e replicabilidade dos ambientes de desenvolvimento. Esta camada é baseada na utilização de containers Docker, que encapsulam todo o ambiente necessário — incluindo o sistema operacional, o ROS, o Gazebo, o MoveIt, além de dependências e bibliotecas auxiliares.

A adoção do Docker permite que os ambientes sejam executados tanto localmente (em qualquer sistema operacional com suporte a containers) quanto remotamente, utilizando GitHub Codespaces ou servidores próprios, sem dependências de configuração do sistema hospedeiro.

- Camada de Middleware Robótico

Esta camada compreende os frameworks principais utilizados no desenvolvimento de aplicações robóticas:

- **ROS (Robot Operating System):** Middleware que provê comunicação entre nós, gerenciamento de sensores, atuadores, drivers e algoritmos de controle.
- **Gazebo:** Simulador 3D de robótica que permite a criação de ambientes virtuais com física realista e integração completa com o ROS.
- **MoveIt!:** Framework de planejamento de movimento, cinemática, detecção de colisões e manipulação de robôs.

Essas ferramentas estão integradas, permitindo que o mesmo código desenvolvido seja utilizado tanto na simulação quanto, futuramente, na operação de robôs reais, com mínima ou nenhuma adaptação.

- Camada de Interface com o Usuário

Para garantir acessibilidade e facilidade de uso, a plataforma oferece diferentes interfaces:

- **VSCode Online:** Ambiente de desenvolvimento baseado em navegador, com suporte a código, terminal e depuração.
- **NoVNC/XFCE:** Ambiente gráfico completo acessível via navegador, que permite executar ferramentas como RViz, Gazebo, terminais e editores gráficos.

Essa camada tem como objetivo aproximar a experiência do usuário da realidade dos ambientes de desenvolvimento profissional em robótica, porém com a simplicidade de acesso via web.

- Camada de Conteúdo Didático

Organiza os materiais produzidos, estruturados em:

- **Tutoriais guiados**, abordando desde conceitos básicos até aplicações avançadas.
- **Atividades práticas**, com roteiros que envolvem programação, simulação e desenvolvimento de algoritmos.
- **Repositórios temáticos**, que podem ser executados de forma independente ou integrados na plataforma, permitindo modularidade e expansão.

d) Codificação dos arquivos de configuração

Nesta etapa foram escritos e testados os arquivos para criação das imagens em Docker - dockefiles, os arquivos para configuração da execução dos containers no VSCode - devcontainer.json e scripts para inicialização do workspace para execução das atividades do ROS. Também foram escritos os arquivos html que compõe o frontend da aplicação hospedada na web (Github pages) e disponibilizada para o acesso:

Nos dockefiles foram definidos:

- Framework ROS:
 - Qual versão e repositório do ROS é utilizado na atividade
 - Pacotes e suas dependências necessárias para execução da atividade
 - Definição dos repositórios para adição de novos pacotes do ROS
 - Instalação dos pacotes para atualização dos pacotes ROS 1 (nova sistemática a partir de junho/2025)
- Ambiente Virtual
 - Configurações de usuário
 - Instalação e configurações do noVNC
 - Instalação e configurações do XFCE + VNC
 - Instalação e configurações do VSCode

- Instalação e configurações do display
- Configuração da interação com os volumes locais (permite que os arquivos editados no container sejam acessados externamente)
- Cópia dos pacotes e ambientes ROS - catkin_ws disponíveis no repositório para o container
- Porta de acesso ao ambiente virtual via noVNC (6080)
- Personalização do ambiente
 - Cópia dos arquivos de scripts e wallpapers a partir do repositório
 - Aplicação dos wallpapers no XFCE4 conforme a atividade em execução
 - Colocação na área de trabalho do XFCE de links para execução de scripts para inicialização do ROS (catkin build do ambiente)

Nos arquivos devcontainer.json foram definidos:

- **Metadados do ambiente:** Nome, qual dockerfile será utilizado, contexto de build
- **Configurações de execução do container:** privilégios, display, volume, uso da memória compartilhada
- **Redirecionamento de portas:** Define a porta 6080 para acesso via noVNC no ambiente gráfico do navegador e 5901 como porta utilizada para acesso via cliente VNC tradicional
- **Configuração das portas:** porta 6080 (noVNC) abre automaticamente no navegador quando a porta é encaminhada e 5901 (VNC) apenas notifica que está disponível
- **Configuração dos volumes:** Monta o diretório local no container e defino onde o workspace estará localizado dentro do container.
- **Extensões do VSCode:** são instaladas automaticamente extensões para programação em Python e C++, desenvolvimento com ROS, Gerenciamento de containers Docker, integração com github, dentre outras
- **Funcionalidades (features):** desktop-lite adiciona o ambiente gráfico XFCE, ativa VNC e noVNC

Ao final deste documento na seção Anexos serão disponibilizados alguns exemplos destes arquivos comentados para referência.

Script de inicialização do serviço noVNC dentro do container - start-vnc.sh

Além dos arquivos de configuração para criação da imagem e execução do container foram necessários scripts para inicialização do ambiente gráfico XFCE dentro do container, o permitindo o acesso remoto por meio de um servidor VNC e também via navegador utilizando o noVNC.

Este script será apresentado comentado na seção de Anexos deste documento.

e) Codificação do frontend da aplicação em HTML

O código HTML elaborado teve como finalidade disponibilizar uma interface acessível, funcional e organizada para apoio ao ensino e à pesquisa na área de robótica, facilitando o acesso a tutoriais, ambientes virtuais e materiais complementares.

Para isso o *frontend* da plataforma LabViR foi codificada seguindo as definições descritas a seguir:

Padrão de desenvolvimento web, utilizando HTML5 para organização dos conteúdos, CSS embutido e o framework TailwindCSS para a definição de estilos e responsividade da página. A organização principal da página contempla elementos como cabeçalho (header), menu de navegação (nav), conteúdo principal (*main*) e rodapé (*footer*).

- Estilo visual baseado em uma estética limpa e responsiva, utilizando recursos do TailwindCSS. O framework permite a definição eficiente de espaçamentos, cores, sombras, tipografia e efeitos visuais. Além disso, são aplicados efeitos de interação, como realce ao passar o cursor e transições suaves, o que contribui para uma navegação mais agradável.
- Navegação é realizada por meio de uma barra superior que fornece acesso rápido às principais seções, incluindo o Guia de Utilização e os Exercícios. O cabeçalho apresenta o nome do laboratório e um subtítulo explicativo que contextualiza o propósito da plataforma.

f) Organização dos conteúdos

O corpo da página oferece uma descrição geral da proposta do LabViR, seus objetivos e instruções para utilização. O conteúdo está organizado em blocos (cards), que representam os diferentes módulos de aprendizagem e os laboratórios virtuais disponíveis. Cada bloco possui uma imagem representativa, um título e um botão que direciona o usuário para a respectiva página do curso ou laboratório.

Foram disponibilizados os seguintes módulos:

- Pré-requisitos;
- ROS Básico 101;
- Manipulação Robótica 101;
- Laboratório Virtual ROS1 (Noetic);
- Laboratório Virtual ROS2 (Jazzy).

Alguns cursos ainda estão em desenvolvimento e aparecem desabilitados na interface, com a indicação 'Em breve', como os módulos de Fundamentos de Robótica e Navegação com ROS.

g) Codificação do material didático

Uma vez finalizadas as configurações do ambiente, o material didático foi codificado utilizando os procedimentos comuns do ROS. A saber:

- Definição de um workspace de trabalho - “catkin_ws” - que contém todos os arquivos necessários para execução da infraestrutura de nós, tópicos, msgs do ROS
- Definição dos pacotes necessários àquela atividade. Isso pode ser feito de várias formas:
 - Obtenção do pacote via github para compilação local via catkin_make ou catkin build;
 - Instalação dos pacotes pré-compilados via apt-get;
 - Criação e desenvolvimento de novos pacotes através do catkin_create_pkg e posterior edição dos arquivos Python ou C++, launch files, config files e outros
- Elaboração dos arquivos de roteiro de atividade em formato markdown (para visualização no ambiente VSCode), arquivos de referência dos pacotes ROS utilizados, gabaritos com respostas e outros arquivos auxiliares para os estudantes executarem as atividades
- Instruções para compilação do pacote e execução do código.

Esse procedimento foi realizado para todas as atividades e projetos disponibilizados na plataforma. Em alguns casos, como nos tutoriais, foram utilizadas outras plataformas para execução dos códigos e atividades com a TheConstructSim. Isso foi realizado para cursos com acesso aberto desta plataforma, conforme será explicado mais adiante neste documento.

6. RESULTADOS E DISCUSSÃO

Esta seção está dividida em duas partes. Na primeira parte serão apresentadas três plataformas com acesso online, que utilizem ferramentas de código aberto e sejam total ou parcialmente de usos gratuito.

Na sequência será apresentada a plataforma de ensino de robótica resultante deste trabalho – LabViR – Laboratório Virtual de Robótica, de forma que possam ser destacadas as contribuições deste trabalho.

1.1. Apresentação das plataformas de ensino de robótica similares

Vários trabalhos tem sido publicados sobre plataformas, kits e metodologias para o ensino de robótica em nível superior (CERVERA; DEL POBIL, 2019), (ANAND *et al.*, 2021), (ALMEIDA; LEÃO; SOUSA, 2024; GONZÁLEZ-GARCÍA *et al.*, 2019; HRANOV; HINOV, [S.d.]; KARALEKAS; VOLOGIANNIDIS; KALOMIROS, 2020a; KRUMINS *et al.*, 2024; PITZER *et al.*, 2012; ROLDÁN-ÁLVAREZ *et al.*, 2023). Parte vai demandar a instalação de softwares auxiliares pelo usuário localmente, outros vão demandar algum tipo de hardware, outros não estarão disponíveis para acesso online. Os trabalhos de (HRANOV; HINOV, [S.d.]; KRUMINS *et al.*, 2024; ROLDAN-ALVAREZ *et al.*, 2023; VROCHIDOU *et al.*, 2018) fornecem um ótimo referencial teórico sobre o tema.

Aqui serão apresentados e comparados três trabalhos que estão em conformidade com os requisitos da plataforma proposta descritos na seção **Requisitos da plataforma**, ou seja:

- Estejam disponíveis online de forma aberta
- Não demandem nenhuma instalação local de softwares localmente
- Não demandem nenhum tipo de hardware para ser executado (embora a inclusão do controle de hardware possa ser futuramente incluída de forma opcional)

Com base nestes critérios foram selecionadas três plataformas:

- TheConstructSim - <https://app.theconstruct.ai/login/>
- Unibotics - <https://unibotics.org/>
- Intel4CoRo/AICOR *Virtual Research Building* VRB - <https://vib.ai.uni-bremen.de/>

1. TheConstructSim

O **TheConstructSim** (TELLEZ, 2017) é uma plataforma de ensino de robótica online que fornece acesso baseado na web a simuladores de robótica como o Gazebo com o suporte ao Robot Operating System (ROS). Ela utiliza o Gzweb, um cliente WebGL para Gazebo, para desenvolver simulações. A plataforma só precisa de um navegador habilitado para WebGL com a maioria dos navegadores modernos. Os navegadores oficialmente suportados são Safari, Chrome, Firefox. Isso significa que ele pode ser usado com qualquer dispositivo que tenha um desses navegadores, incluindo Linux, Windows, MacOS ou até tablets e smartphones.

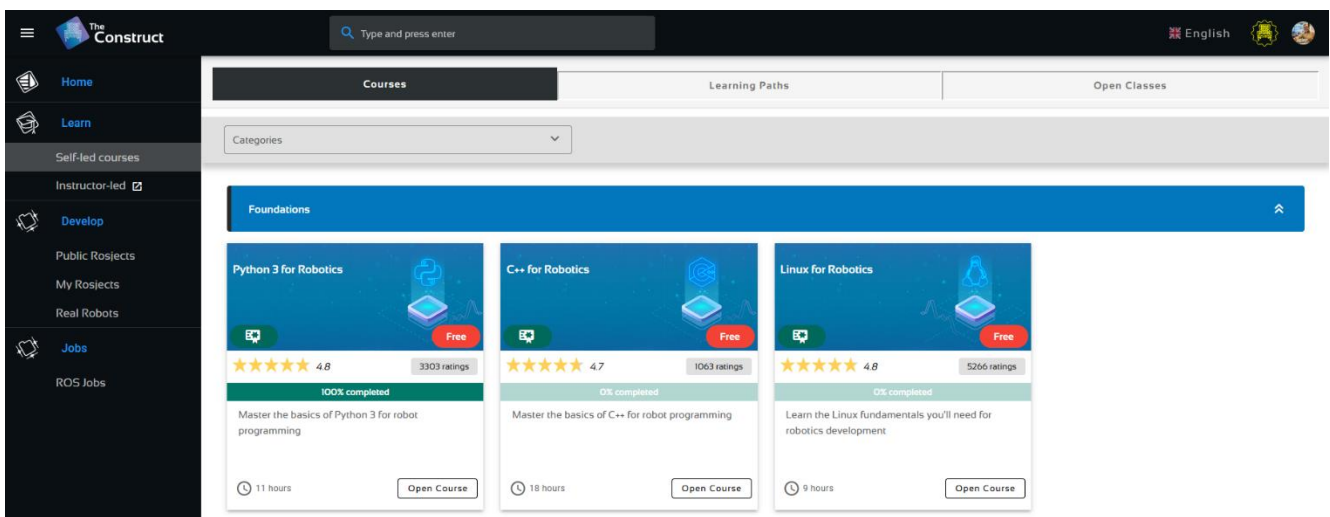


Figura 13-Página inicial da plataforma TheConstructSim

Em termos de recursos educacionais, o TheConstructSim disponibiliza dezenas de cursos estruturados por níveis e temas — ROS 1/2, URDF, OpenCV, drones, manipuladores, integração web, teste unitário, CI/CD, segurança, entre outros além de trilhas de aprendizado orientadas.

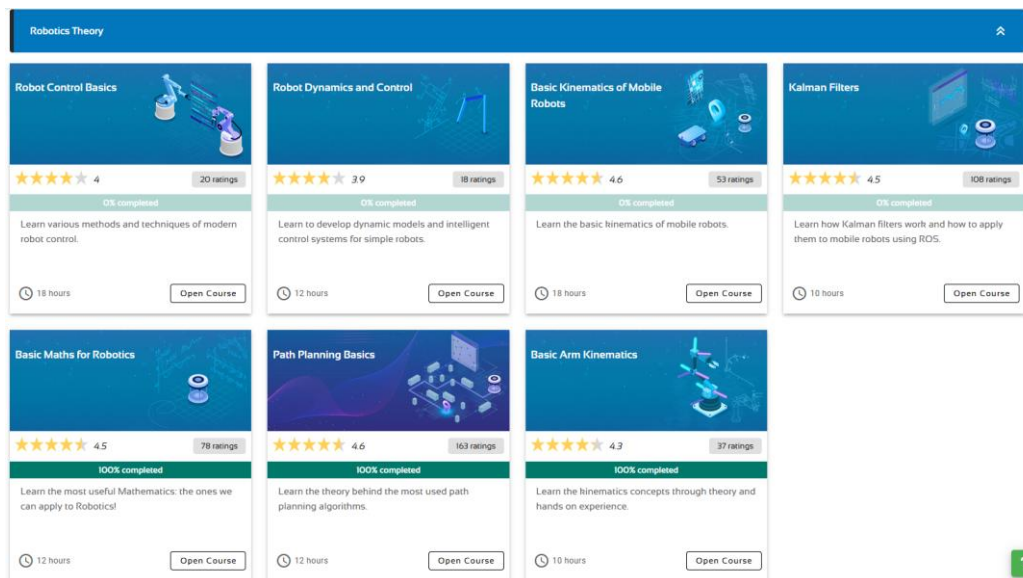


Figura 14-Portfólio de cursos de Teoria de Robótica no TheConstrucSim

Conhecida como provedor oficial de treinamento em ROS, possui reconhecimento global e parcerias com grandes empresas e instituições. O acesso a plataforma é pago e pode ser de forma individual ou institucional (através de convênios com entidades educacionais, empresas), mas o acesso também é disponibilizado de forma gratuita para alguns cursos e acesso à área de projetos (ROSjects). Atualmente os seguintes cursos são gratuitos e abertos à comunidade:

- Python 3 para robótica
- C++ para robótica
- Linux para robótica

Estes cursos foram incluídos na plataforma proposta neste trabalho por serem gratuitos e fornecerem os conhecimentos necessários para iniciantes na programação de sistemas robóticos com o ROS.

2. UniBotics

A plataforma Unibotics (ROLDÁN-ÁLVAREZ *et al.*, 2023) é um ambiente online aberto para o ensino prático de robótica, especialmente desenvolvido para a educação superior. Sua concepção surge da necessidade de oferecer um meio eficiente e acessível para que estudantes possam aprender robótica de forma prática, superando as dificuldades comuns de instalação de softwares complexos e de acesso a laboratórios físicos. A plataforma permite que os alunos programem e executem códigos diretamente do navegador, utilizando tecnologias amplamente empregadas na indústria, como o simulador Gazebo e o middleware ROS.

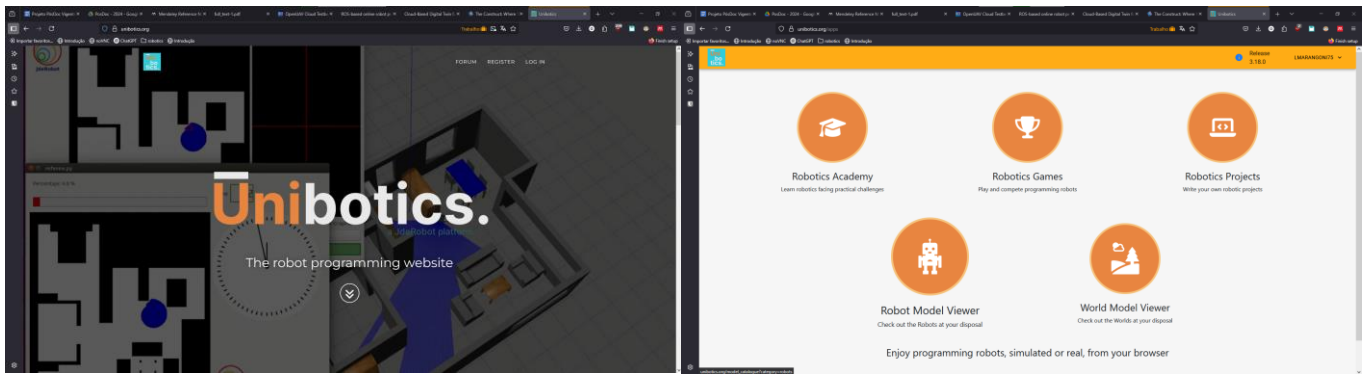


Figura 15 - Plataforma Unibotics Página de entrada e página de acesso aos recursos

A arquitetura da plataforma **Unibotics** é composta por um ecossistema que integra diversas tecnologias para oferecer um ambiente de desenvolvimento e simulação de robôs diretamente pelo navegador. Seu núcleo é baseado na imagem Docker chamada **RADI (Robotics Academy Docker Image)**, que encapsula todo o software necessário, incluindo o simulador Gazebo e o middleware ROS, além das dependências específicas para cada exercício. Isso permite que tanto o simulador quanto o código do aluno rodem localmente, distribuindo a carga computacional e tornando a plataforma altamente escalável. A interface gráfica é acessada via navegador, onde o usuário interage com os exercícios, edita o código, executa a simulação e recebe informações visuais, como imagens da câmera do robô, status dos sensores e a cena simulada.

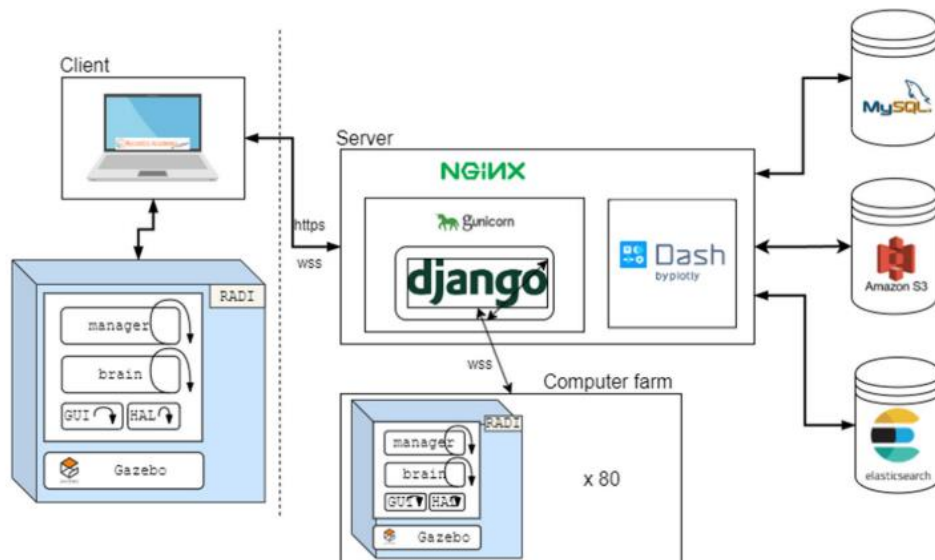


Figura 16 - Arquitetura da plataforma Unibotics(ROLDÁN-ÁLVAREZ et al., 2023)

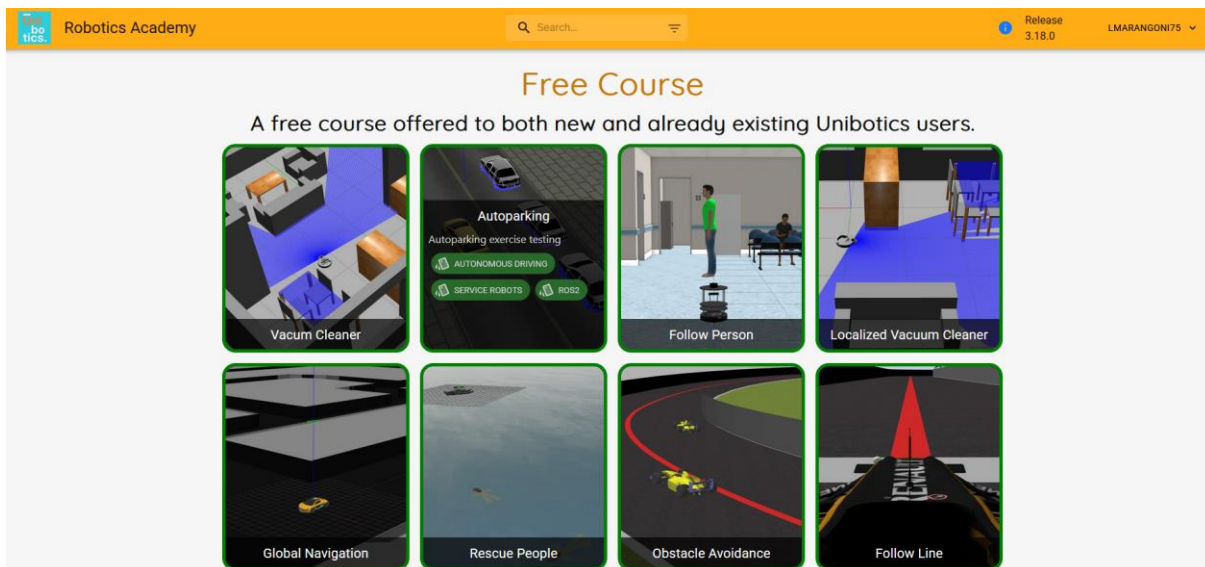


Figura 17 - Plataforma Unibotics, Robotics Academy Cursos disponíveis gratuitamente

Para execução realização dos cursos é necessário que o usuário tenha o Docker instalado em sua máquina e baixe e execute a imagem do chamado “*Robotic Backend*” a partir do DockerHub.

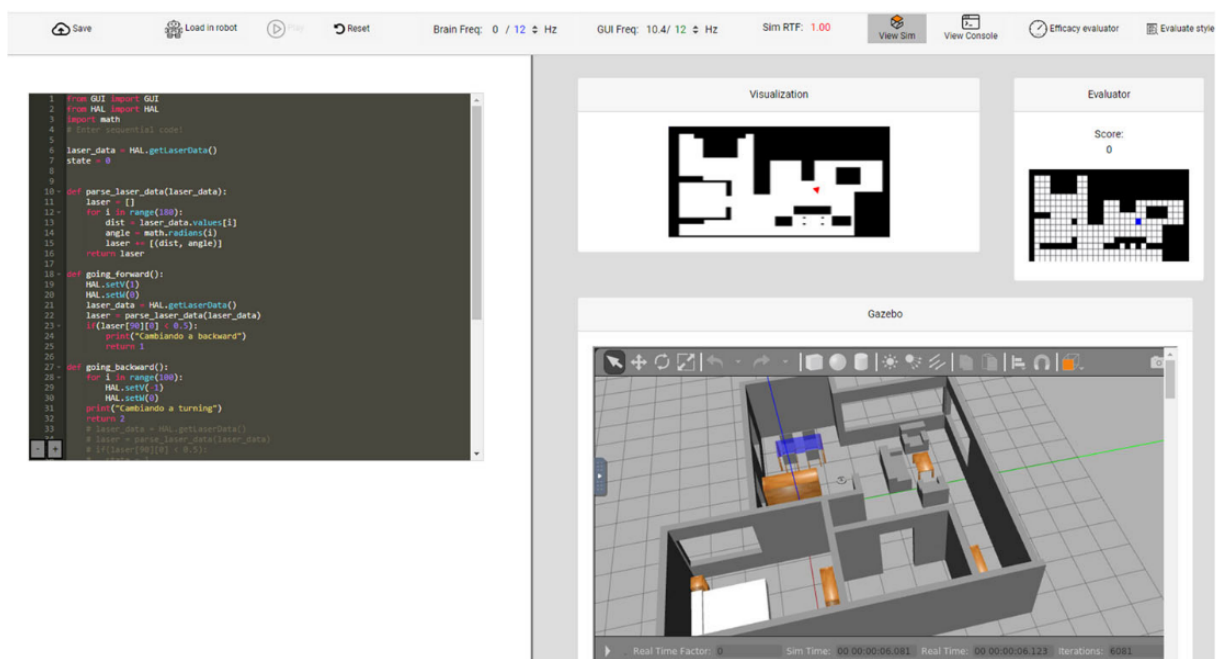


Figura 18 - Plataforma Unibotics. Página do curso com o aspirador robô (ROLDÁN-ÁLVAREZ et al., 2023)

O sistema é orquestrado por um servidor web desenvolvido com o framework **Django**, responsável por gerenciar as conexões, servir os exercícios, armazenar os códigos dos usuários (em uma infraestrutura baseada no Amazon S3) e registrar as atividades dos usuários em bancos de dados MySQL e Elasticsearch. A plataforma possui uma ferramenta de visualização de dados, construída com o framework **Dash**, que permite monitorar o uso da plataforma e o desempenho dos alunos,

oferecendo métricas como tempo de uso, progresso nas tarefas e qualidade do código. Para alunos que não conseguem rodar o Docker em seus próprios computadores, a Unibotics oferece uma **fazenda de 80 computadores** na universidade, que executam as instâncias da RADl sob demanda, garantindo acesso total às funcionalidades sem a necessidade de instalação local.

Entre os principais recursos educacionais, a Unibotics oferece mais de 20 exercícios práticos que abrangem temas como robótica móvel, drones, direção autônoma e visão computacional. As atividades são projetadas como desafios, onde os alunos devem resolver problemas reais, como navegação autônoma, mapeamento, localização, inspeção com drones e reconstrução 3D.

3. Intel4CoRo/AICOR Virtual Research Building VRB

A plataforma Intel4CoRo é uma plataforma de robótica aberta e baseada em nuvem para ensino e treinamento de conceitos de robótica cognitiva (referência). Ela utiliza o serviço de código aberto BinderHub (referência) para implantar e operar aplicações containerizadas, incluindo ambientes de simulação robótica e coleções de software baseadas no Robot Operating System (ROS).

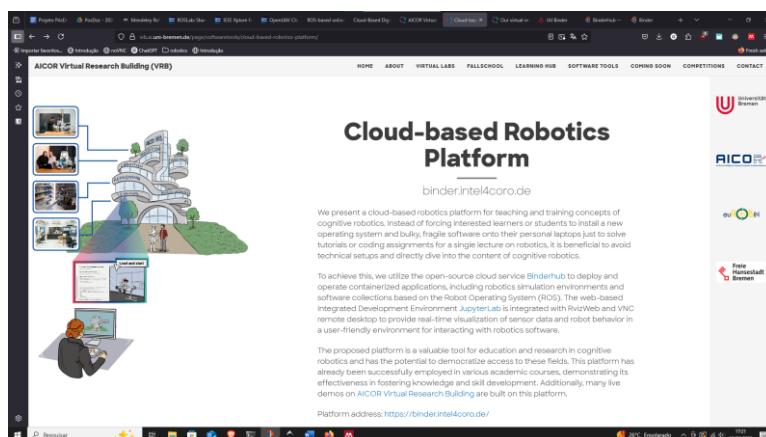


Figura 19 - Plataforma Binder.Intel4CoRo página de apresentação

O ambiente de desenvolvimento integrado baseado na web com JupyterLab (PROJECT JUPYTER, 2025), é integrado com RvizWeb e com um desktop remoto via VNC, proporcionando visualização em tempo real dos dados dos sensores e do comportamento dos robôs, em um ambiente amigável para interação com o software de robótica.

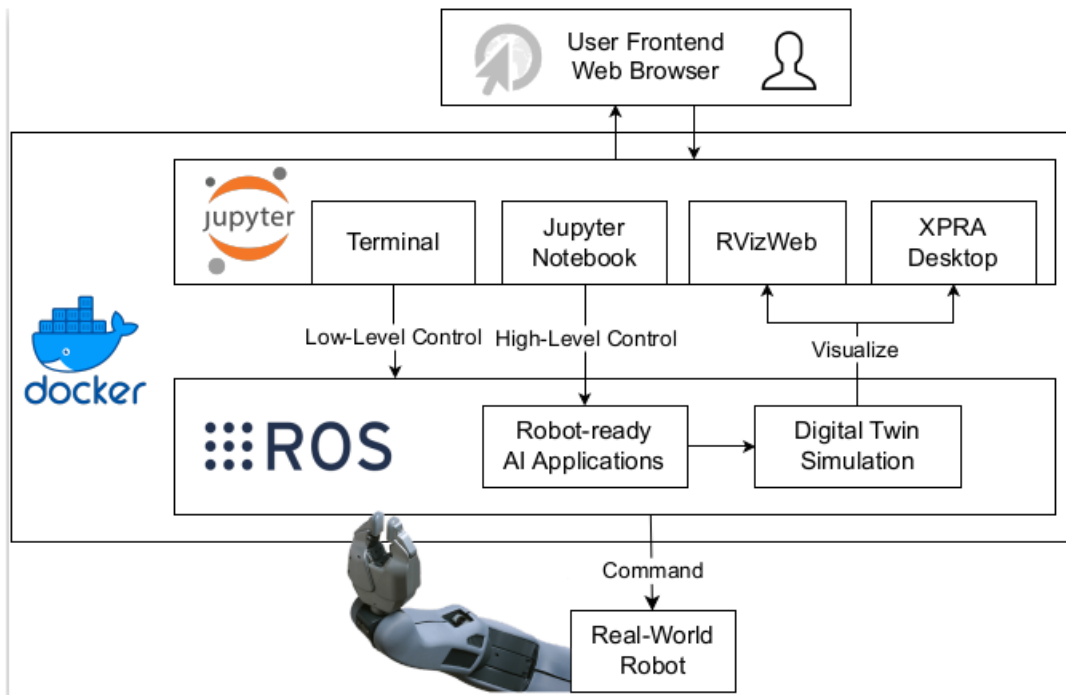


Figura 20 – A plataforma permite o controle de robôs no mundo real e a simulação de gêmeos digitais com o ROS e as ferramentas de integração dentro do Docker contêiner (NIEDŹWIECKI et al., 2024)

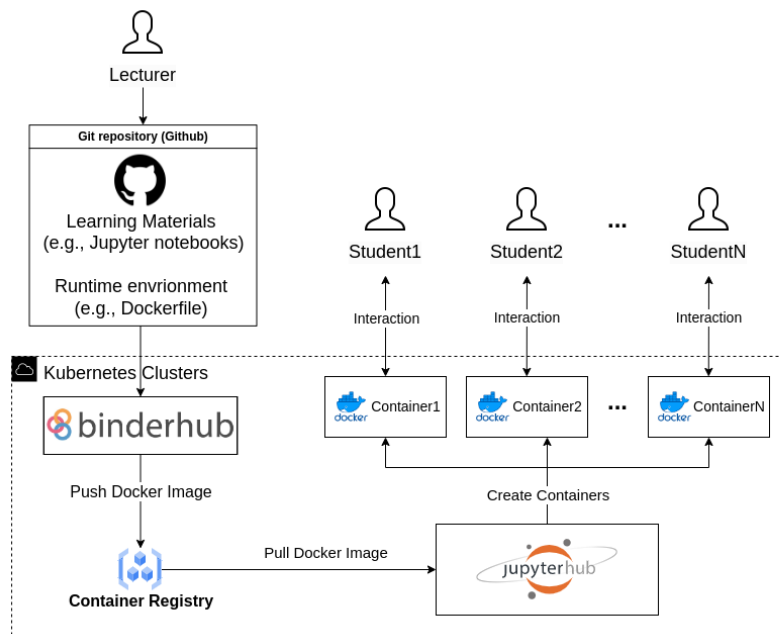


Figura 21 – Framework do serviço em nuvem montado para o VRB. O BinderHub inicia um novo container para cada usuário conectado. (NIEDŹWIECKI et al., 2024) e faz toda a orquestração utilizando o Kubernetes Clusters

A plataforma é aberta para a criação e execução de containers customizados para aplicações de robótica. Para isso o instrutor deve criar um repositório conforme as orientações da plataforma e

disponibilizar no github. A partir daí o material fica disponível para os alunos baixarem em seus próprios computadores e executarem através do navegador web.

Um exemplo de aplicação desta plataforma é o AICOR *Virtual Research Building* (VRB) que é uma plataforma virtual baseada no Intel4CoRo e BinderHub que funciona como um centro de pesquisa e desenvolvimento voltado para robótica cognitiva e inteligência artificial. Dentro do VRB, os usuários podem acessar laboratórios de pesquisa de ponta, incluindo ambientes em desenvolvimento e espaços dedicados a componentes de software. A plataforma oferece um livro didático interativo, que combina fundamentos teóricos com exercícios práticos de programação em laboratórios virtuais.

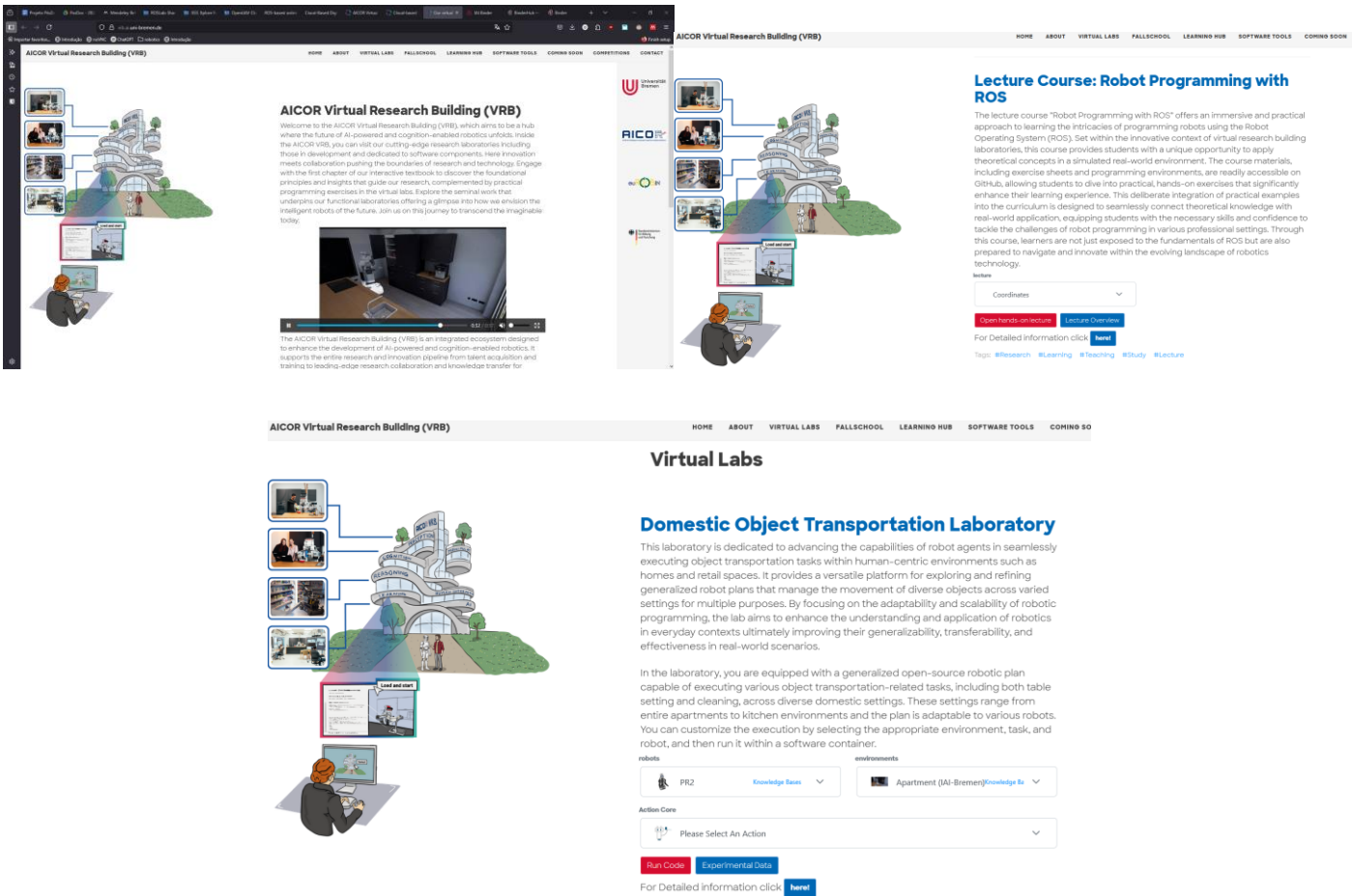


Figura 22 - Virtual Research Building - VRB - Laboratórios Virtuais em Robótica Cognitiva. Página de apresentação, Curso de Programação com ROS e Laboratório Virtual de Transporte de Objetos Domésticos

1.2. Apresentação da plataforma LabViR

Nesta seção será apresentada a plataforma LabViR – Laboratório Virtual de Robótica disponível para acesso no endereço <https://nuria-ifsp.github.io/website/>. A concepção da plataforma ocorreu seguindo o mapa mostrado na Figura 23

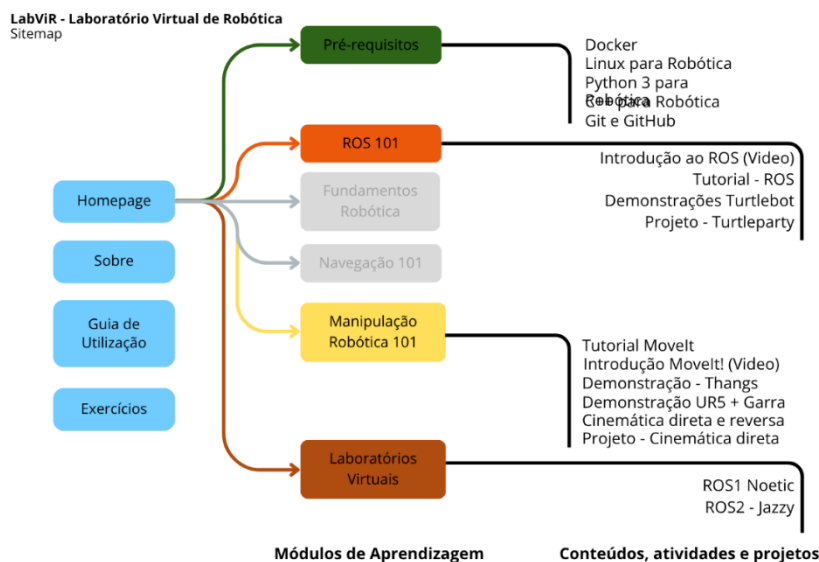


Figura 23 - Sitemap da plataforma LabViR

A página de entrada - *homepage* - apresenta informações sobre a plataforma, guia de utilização – com orientações sobre como realizar as atividades e acesso rápido a exercícios e projetos, conforme Figura 24.



Figura 24 - Página de entrada da plataforma

O conteúdo foi organizado na forma de módulos de aprendizagem, conforme mostrado na Figura 25. Os módulos de aprendizagem inicialmente criados são o de pré-requisitos, Introdução ao ROS – ROS Básico 101, Manipulação robótica 101 e Laboratórios Virtuais para ROS 1 e 2. Os módulos fundamentos de robótica e Navegação com ROS 101 são previstos para inclusão em breve.

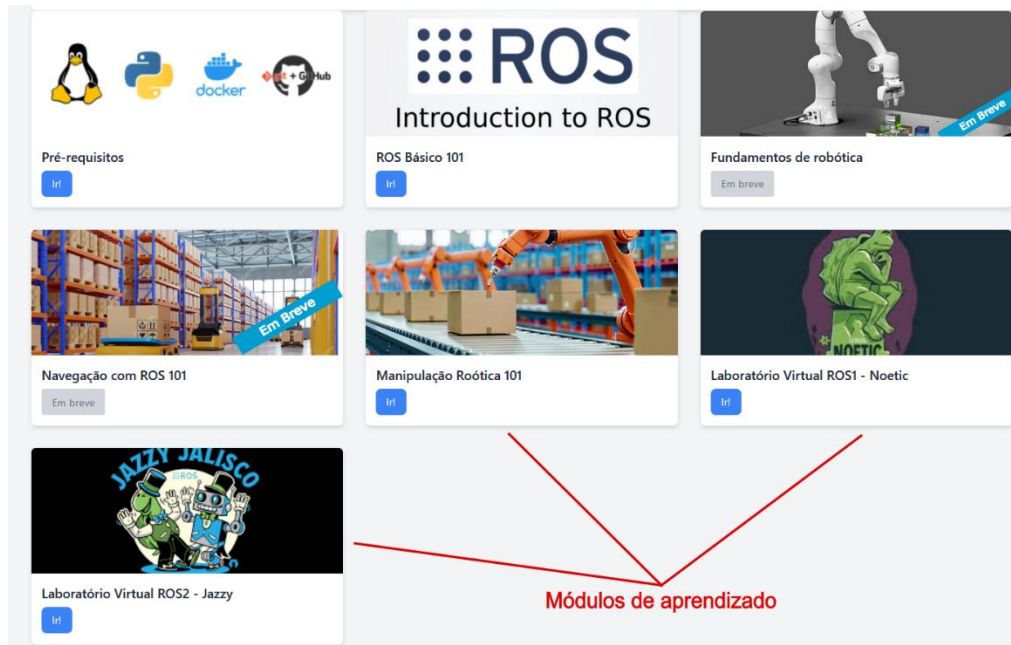


Figura 25 - Módulos de aprendizagem da plataforma LabViR

Descrição dos módulos e cursos da plataforma

- **Módulo Pré-requisitos:** Para acesso às atividades de robótica é importante que os alunos tenham proficiência mínima nos seguintes tópicos:
 - **Linux para Robótica** – Os ambientes virtuais criados localmente ou na nuvem são implementados em ambiente Linux Ubuntu (20.04 para o ROS 1 ou 24.04 para o ROS 2). Por conta disso é fundamental que os estudantes saibam trabalhar neste ambiente
 - **Python 3 para Robótica** – O Python é uma das linguagens que podem ser utilizadas para o desenvolvimento dos nós (nodes) do ROS. Grande parte dos exemplos, tutoriais e exercícios é disponibilizada em Python e fazem o uso dos conceitos de Programação Orientada a Objetos (POO). Neste curso o estudante aprende a utilizar o Python 3 a partir de exemplos com o ROS
 - **C++ para Robótica** – Utilizado no ROS para o desenvolvimento dos nós (nodes) e também para a implementação de plugins que realizam a interface entre os nós do ROS e dispositivos externos (sensores, atuadores). A programação em C++ é um requisito quando são realizadas atividades relacionadas a algum tipo de hardware.
 - **Docker** – A organização dos workspaces de desenvolvimento na forma de containers é um dos pontos-chave para a virtualização dos ambientes e permite que os ambientes sejam pré-configurados e

distribuídos mais facilmente. O conhecimento dos conceitos do Docker, dockerfiles, devcontainers entre outros possibilita aos alunos a customização dos repositórios disponibilizados neste trabalho para inclusão do próprio conjunto de pacotes em função do trabalho realizado.

Os cursos de Linux, Python 3 e C++ foram incorporados utilizando a infraestrutura do site TheConstructSim em que tais cursos são gratuitos, bastando para isso que o aluno faça um cadastro no site. A Figura 26 mostra o módulo de aprendizagem de Pré-requisitos com os cursos oferecidos.

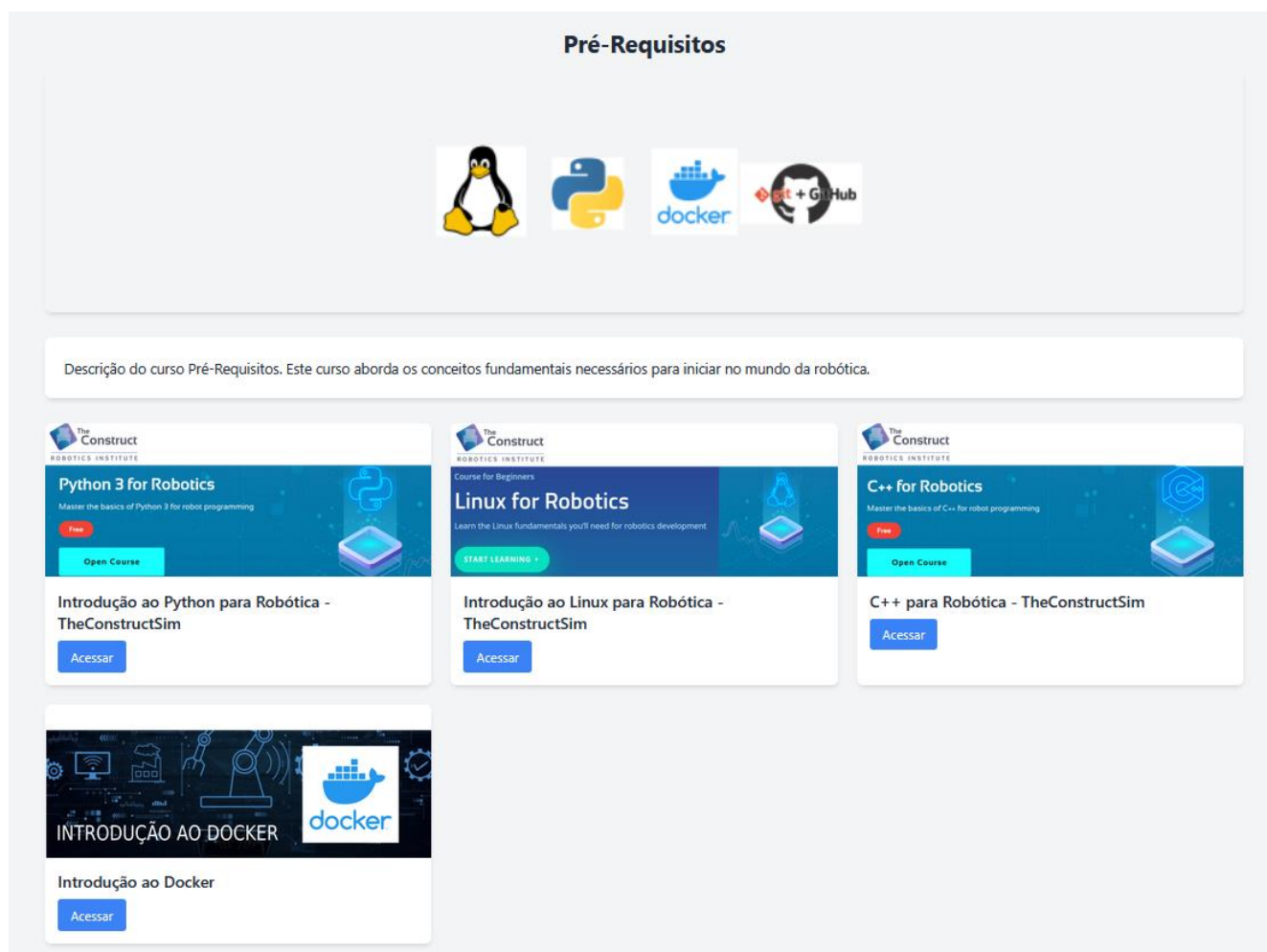


Figura 26 - LabViR - Módulo de aprendizagem com os Pré-requisitos para programação em robótica

Módulo Introdução ao ROS (ROS 101): Para que os alunos possam desenvolver projetos robóticos no contexto dos ambientes virtuais é fundamental que tenham conhecimento e proficiência no ROS. Este módulo fornece os subsídios para que os alunos possam adquirir esta proficiência e está dividido da seguinte forma:

○ Tutoriais

- *Curso Introdução ao ROS* – conjunto de videoaulas em português sobre o tema.
- *Tutoriais oficiais do ROS* – Link de acesso ao site oficial do ROS. Ponto de partida para aprendizagem do ROS em nível básico, intermediário e avançado. Com atividades passo-a-passo e descrição de conceitos em português
- *Repositório ROS – Tutorials* – repositório que pode ser baixado a partir do GitHub com todos os exercícios do tutorial do ROS do site oficial resolvidos. Este repositório pode ser carregado no VSCode no ambiente de um contêiner e os exemplos podem ser estudados e executados.

○ Demonstrações

- *Nestas atividades são apresentados três exemplos de projetos básicos com o ROS utilizando o pacote Turtlebot que simula o controle de um robô tartaruga em ambiente bidirecional utilizando os conceitos do ROS – nós, tópicos, parâmetros, ações, serviços, criação de pacotes, entre outros. As demonstrações são ricamente comentadas e devem ser executadas em sequência apresentada.*

○ **Projetos** – Para o encerramento deste módulo é disponibilizado um desafio chamado TurtleParty, baseado no pacote Turtlebot. Arquivos iniciais são fornecidos e o estudante devem realizar o desafio conforme o enunciado proposto. Para realização do projeto o aluno deve se basear nas demonstrações apresentadas anteriormente.

A Figura 27 mostra este módulo de aprendizagem na plataforma.

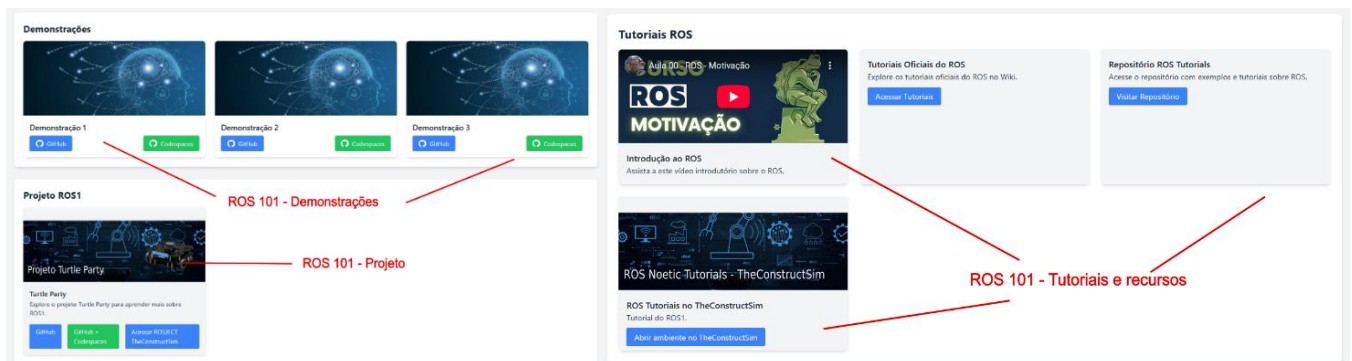


Figura 27 - Módulo de aprendizagem do ROS1 com tutoriais, demonstrações e projeto final

Neste ponto é importante mostrar como é realizada a opção de execução da atividade localmente ou em nuvem. Cada um dos cursos/atividades deste módulo possui a configuração mostrada na Figura 28.

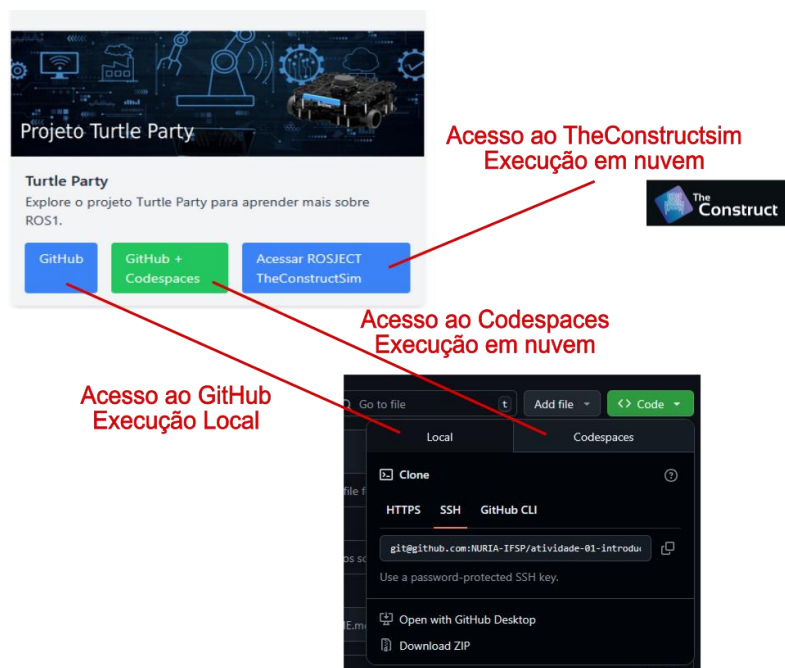


Figura 28 – Unidade de cada curso/projeto do módulo com botões de acesso ao GitHub, Codespaces ou TheConstructSim que possibilita a realização do exercício em ambiente local ou em nuvem (Codespaces ou TheConstructSim)

Uma vez estando conectado ao github.com ao acessar na unidade do curso o aluno pode escolher entre as seguintes opções:

- **Github** – Acesso ao repositório no Github que pode ser baixado pelo estudante e executado localmente no VSCode em ambiente Linux (preferível) ou Windows com Docker instalado. O aluno fica com uma cópia do repositório em seu computador e pode modifica-lo conforme seu projeto sem restrições de tempo de utilização e tamanho do contêiner;
- **Codespaces** – Acesso à nuvem do Codespaces. Ao acessar o link o Codespaces irá criar a imagem da aplicação remotamente na área do estudante no GitHub (por isso há necessidade de cadastro anterior) e irá executar o contêiner remotamente. Uma aba do navegador deverá abrir com o VSCode executado em nuvem, e o estudante deverá acessar a partir do container a porta para o ambiente virtual de desenvolvimento (aba “Ports” do VSCode). Ao clicar no link exibido para a porta 6080 uma nova aba será aberta no navegador com o ambiente virtual noVNC + XFCE com todas as configurações para execução da atividade. Neste caso é fundamental que após utilizar o ambiente o usuário encerre a execução do container no

VSCode o tempo disponível por mês é limitado a 60hs mensais no Codespaces (ou 80hs se for estudante).

- **TheConstructSim** – Algumas atividades são disponibilizadas para utilização a partir do ambiente virtual disponibilizado pelo TheConstructSim. Embora o ambiente seja bastante integrado, há a restrição de utilização de 8h mensais nesta plataforma.
- **Módulo: Introdução Manipulação Robótica – Manipulação 101:** Diversas são as áreas possíveis para o desenvolvimento de projetos em ambiente virtual com o ROS. Optamos pela inclusão de aplicações práticas na área de manipulação robótica utilizando para isso o pacote MoveIt!.

Este módulo de aprendizagem foi incorporado com a seguinte estrutura:

- **Tutoriais:**
 - Videoaula (em inglês) explicando os fundamentos da ferramenta MoveIt
 - Tutorial MoveIt! – Tutorial passo-a-passo da página oficial da ferramenta para execução no ambiente virtual localmente ou em nuvem
- **Demonstrações**
 - **Demonstração 1 – Braço robótico Thangs** – aqui é demonstrado como deve ser realizada a modelagem do atuador robótico Thangs (THANGS, [S.d.]), bastante popular e encontrado em vários projetos de manipuladores robóticos impressos em 3D, no ROS e o seu controle e simulação com o ROS, Gazebo e o MoveIt!
 - **Movimentação do UR5 com garra Robotiq** – Demonstração de um projeto do tipo Pick-and-Place com o robô universal UR5 e garra Robotiq. Esta configuração de robô e garra é bastante utilizada para simulações com o MoveIt em ambientes ROS por ser bastante comum em ambientes industriais reais. Nesta demonstração o aluno deve realizar a operação pick-and-place fornecendo as coordenadas das juntas do robô – cinemática direta através de um script de movimentação do robô e controle da garra. Não há planejamento da rota com o MoveIt! De forma que esta atividade demonstra as dificuldades para programação da movimentação do manipulador de forma direta.
 - **Utilização do MoveIt! Cinemática reversa com o UR10** – Nesta demonstração a movimentação do manipulador robótico é realizada pelo MoveIt! Através de planejamento da rota e cálculos de cinemática reversa realizados pela ferramenta. Uma operação de Pick-and-place deve ser planejada no ambiente do MoveIt! Com o UR10 com a garra PG-70 (configuração bastante comum em ambientes industriais).
- **Projetos:** São propostos dois projetos como sequencia das demonstrações apresentadas anteriormente, um de cinemática direta com o UR5 e outro de cinemática reversa com o UR10 utilizando o MoveIt!

A Figura 29 mostra como foi implementado o módulo na plataforma.

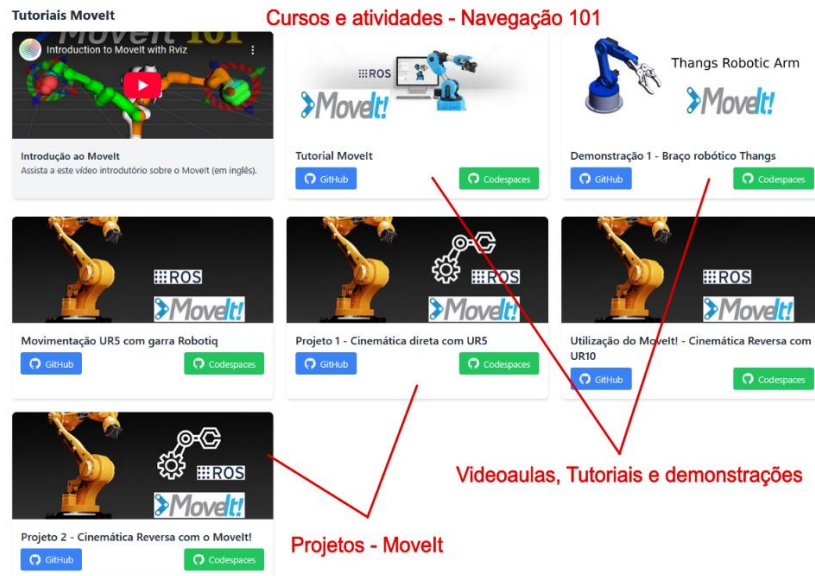


Figura 29 - Módulo de aprendizagem de Manipulação Robótica com os recursos, atividades e projetos disponibilizados

Laboratórios Virtuais para o ROS1 e ROS2

Com o objetivo de fornecer um ambiente de desenvolvimento pré-configurado para o desenvolvimento de aplicações robóticas foram incorporados à plataforma as unidades de desenvolvimento para o ROS 1, na última versão LTS, a Noetic Ninjems, e na versão LTS atual para o ROS 2 – Jazzy Jalisco.

Ambos os ambientes foram configurados com a opção de imagem “desktop-full” que incorpora uma instalação bastante completa do ROS, seus principais pacotes e ferramentas como o RViz, Gazebo, rqt entre outras.

Estas unidades servem de ponto de partida para o desenvolvimento de projetos robóticos com o ROS, bastando para isso a instalação dos pacotes adicionais referentes a cada projeto específico. O que, sem dúvida, agiliza bastante o desenvolvimento de projetos robóticos com o ROS. Sua implementação na plataforma é mostrada na Figura 30.

Laboratórios Virtuais ROS 1 e 2

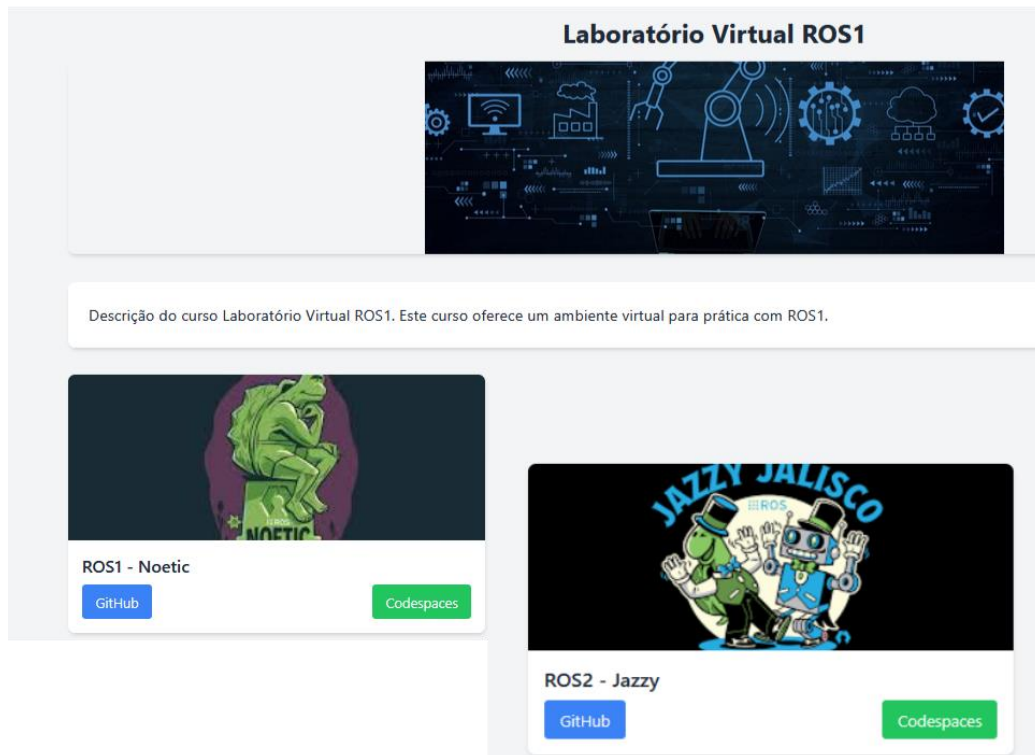


Figura 30 - Laboratórios virtuais disponibilizados no LabViR com o ROS 1 - Noetic e ROS 2 - Jazzy

Por fim, será apresentada a área da plataforma dedicada à inclusão de atividades e projetos. Nesta área, acessível a partir da página principal da plataforma, o aluno tem acesso a unidades com exercícios e projetos disponibilizados na plataforma pelos instrutores/docentes. Nesta versão inicial foram disponibilizados exercícios com o ROS e projetos de movimentação robótica com cinemática direta e inversa com o MoveIt, conforme mostrado na figura



Figura 31 - Área de exercícios e projetos acessível a partir da homepage do projeto. Aqui podem ser disponibilizados projetos e desafios pra os estudantes de forma que possam ser facilmente encontrados.

7. Discussão dos resultados

A Tabela 2 apresenta uma comparação das plataformas de ensino de robótica TheConstructSim, Unibotics e AICOR/VRB com a plataforma proposta neste trabalho, com relação aos requisitos definidos.

Tabela 2 - Comparação das plataformas de ensino de robótica TheConstructSim, Unibotics, AICOR/VRB e LabViR

	Requisito	TheConstructSim	Unibotics	AICOR VRB	LABVIR
1	Acesso aberto, sem necessidade de login	Não	Não	Sim	Sim
2	Acesso gratuito	Não	Sim	Sim	Sim
3	Tempo mínimo de setup para o aluno iniciante	Sim	Não	Sim	Sim
4	Necessidade de instalação de softwares na máquina local	Não	Sim	Não	Não
5	Simulação em ambiente Virtual	Sim	Sim	Sim	Sim
6	Simulação em ambiente Local	Não	Não	Sim	Sim
7	Customizável	Parcialmente	Sim	Sim	Sim
8	Possibilita o controle de robôs reais	Sim	Não	Sim	Não
9	Demandam a manutenção de um servidor próprio	Não	Sim	Sim	Não

No que diz respeito ao **acesso aos conteúdos** (requisito 1), a plataforma **LabViR** foi concebida para maximizar a acessibilidade tanto para alunos iniciantes quanto para instrutores. Por essa razão, todo o acesso ocorre diretamente via navegador, sem a exigência de criação de contas, usuários ou senhas. Das plataformas comparadas, apenas **LabViR** e **AICOR/VRB** oferecem essa facilidade.

Em relação à **gratuidade** dos conteúdos (requisito 2), observa-se que a plataforma **The ConstructSim**, embora possua alguns recursos gratuitos, opera em modelo comercial, com diversas funcionalidades restritas a assinantes. As demais plataformas, incluindo a **LabViR**, disponibilizam seus conteúdos de forma totalmente aberta e gratuita para a comunidade.

Nos requisitos de **facilidade de acesso e configuração inicial** (requisitos 3 e 4), destaca-se que os usuários da **LabViR** não precisam realizar qualquer instalação ou configuração local, bastando acesso a um navegador e a uma conta no GitHub. Situação semelhante ocorre nas plataformas **The**

ConstructSim e **AICOR/VRB**. Por outro lado, a **Unibotics** requer que os alunos instalem localmente um contêiner Docker específico (denominado **RADI**) para que possam acessar o ambiente remoto.

Quanto à **execução de simulações em ambiente virtual** (requisito 5), todas as plataformas analisadas atendem plenamente. Entretanto, no que se refere à **execução local das simulações** (requisito 6), apenas a **LabViR** e a **AICOR/VRB** oferecem essa possibilidade. Isso permite que os usuários baixem os ambientes e os executem localmente, utilizando seus próprios recursos computacionais, desde que possuam o Docker instalado, preferencialmente em sistemas operacionais baseados em Linux. Essa funcionalidade é particularmente relevante para atividades que demandem maior poder de processamento ou customização do ambiente de simulação, superando as limitações impostas pelos ambientes em nuvem.

No que se refere à **possibilidade de personalização** (requisito 7), a **LabViR** adota uma abordagem totalmente aberta, estruturando seus conteúdos em repositórios públicos no GitHub. Esses repositórios incluem tanto os materiais didáticos (aulas, atividades, tutoriais) quanto os arquivos necessários para a criação dos ambientes virtuais via contêineres Docker. Esse modelo facilita a adaptação e expansão da plataforma, permitindo que usuários criem novos módulos, unidades de aprendizagem e adaptem os ambientes às suas necessidades específicas. De modo similar, **Unibotics** e **AICOR/VRB** oferecem alto grau de personalização, especialmente quando executadas em ambientes próprios. Já a **The ConstructSim** permite customização por meio de seus projetos em ROS, denominados **ROSjects**, que integram ROS e Jupyter Notebooks.

Em relação à **infraestrutura de hospedagem e manutenção** (requisito 9), tanto **Unibotics** quanto **AICOR/VRB** demandam a gestão de servidores próprios. A **Unibotics**, por exemplo, mantém uma infraestrutura robusta composta por mais de 80 servidores (ROLDÁN-ÁLVAREZ *et al.*, 2023), enquanto a **AICOR/VRB** está hospedada na Universidade de Bremen, na Alemanha. A **The ConstructSim**, por ser uma plataforma comercial, também opera com infraestrutura própria, sediada na Espanha. Diferentemente, a proposta da **LabViR** busca minimizar a necessidade de infraestrutura, adotando serviços em nuvem como o **GitHub Codespaces**, o que simplifica significativamente sua implementação e manutenção. Embora essa abordagem gere uma dependência em relação a serviços externos, ela reduz custos e viabiliza sua adoção por instituições com recursos limitados ou por grupos de pesquisa menores.

Comparando-se às plataformas analisadas, a **LabViR** apresenta características que favorecem sua adoção para ensino e pesquisa na área de robótica, especialmente pela **facilidade de acesso, ausência de custos, rapidez na implementação e não exigência de infraestrutura própria**. As atividades podem ser iniciadas quase imediatamente, bastando que o usuário possua uma conta no GitHub. Além disso, a disponibilização dos ambientes em repositórios permite não apenas a utilização

em nuvem, mas também sua execução local, com a vantagem de acesso a recursos computacionais mais robustos, quando necessário.

Por outro lado, a utilização de serviços de terceiros, como o **GitHub Codespaces**, traz uma limitação: a dependência de um serviço externo que não está sob controle dos desenvolvedores da plataforma. No entanto, essa estratégia se mostrou vantajosa por possibilitar uma implementação rápida, com configuração simplificada. Alternativas foram consideradas, como o **MyBinder.org** (BINDER.ORG, 2025), utilizado pela **AICOR/VRB**, mas essa opção exigiria adaptações mais complexas, como a execução do ROS diretamente em Jupyter Notebooks — uma abordagem que, neste momento, foi considerada menos prática. Contudo, essa alternativa permanece válida para estudos futuros, visando à redução da dependência de serviços externos.

Por fim, cabe destacar que a **LabViR** não contempla, até o momento, o **controle de hardware real**. A inclusão dessa funcionalidade, embora possível, implicaria o aumento considerável da complexidade da plataforma, exigindo desenvolvimento de sistemas para transmissão de vídeo em tempo real, agendamento de uso de robôs e manutenção de infraestrutura dedicada. Tal abordagem contraria a filosofia do projeto, que prioriza a **simplicidade, acessibilidade e baixo custo operacional**.

8. CONCLUSÕES

Este projeto propôs o desenvolvimento de uma **plataforma colaborativa para ensino e pesquisa em robótica**, que visa favorecer a utilização, o compartilhamento e a disseminação de conteúdos, atividades e projetos entre grupos de pesquisa e ensino. A proposta busca ampliar o acesso a recursos tecnológicos que fomentem o desenvolvimento e a inovação na área de robótica.

Para embasar sua concepção, foi realizado um **estudo sistemático de plataformas similares**, disponíveis na literatura e na prática, com o objetivo de compreender seus modelos, requisitos, arquiteturas de sistema e os recursos necessários para sua implementação e operação.

A partir desse estudo, foi desenvolvida a **plataforma LabViR**, disponibilizada de forma aberta na internet, estruturada em módulos de aprendizagem temáticos, cada um contendo cursos, atividades e recursos voltados ao ensino e desenvolvimento de aplicações robóticas. As unidades são organizadas como repositórios no GitHub, que podem ser **executados tanto localmente, mediante o uso de Docker, quanto remotamente por meio do ambiente em nuvem do GitHub Codespaces**. Esta abordagem elimina a necessidade de instalação prévia de softwares, reduzindo barreiras de acesso e facilitando a replicação dos ambientes. Além disso, permite a personalização dos repositórios e dos setups, o que é particularmente útil em contextos de pesquisa que exigem ambientes complexos, com múltiplos pacotes, bibliotecas e dependências específicas.

Os resultados alcançados com este projeto apontam que a plataforma LabViR oferece uma solução viável, acessível e escalável para o ensino e pesquisa em robótica, alinhada aos princípios de disseminação do conhecimento aberto e de redução das barreiras técnicas de acesso.

Adicionalmente, a plataforma abre caminho para contribuições futuras em três frentes principais:

- **Aperfeiçoamento da plataforma**, por meio da incorporação de novos ambientes de desenvolvimento, módulos de aprendizagem, ferramentas e funcionalidades.
- **Desenvolvimento de aplicações em robótica**, possibilitando a integração de pacotes avançados, como navegação, mapeamento (SLAM), visão computacional, manipulação e outras tecnologias.
- **Pesquisa e desenvolvimento de algoritmos e tecnologias aplicadas**, utilizando o framework ROS como base para experimentação, testes e validação de soluções, fomentando a colaboração entre pesquisadores e desenvolvedores da comunidade.

Como desdobramento deste trabalho, estão previstas ações de extensão e disseminação, especialmente no âmbito do **IFSP e da Unicamp**, com a realização de atividades formativas para grupos de pesquisa, bem como a promoção de projetos de iniciação científica em temas como navegação autônoma, SLAM e percepção robótica. Além disso, está em andamento a preparação de publicações para congressos e periódicos da área de robótica, com ênfase em metodologias e ferramentas para a educação em robótica.

9. Referências Bibliográficas

ALMEIDA, Filipe; LEÃO, Gonçalo; SOUSA, Armando. An Educational Kit for Simulated Robot Learning in ROS 2. *Lecture Notes in Networks and Systems*, v. 978 LNNS, p. 513–525, 2024. Disponível em: <https://doi.org/10.1007/978-3-031-59167-9_42>. Acesso em: 1 abr. 2025.

ANAND, Harish *et al.* OpenUAV Cloud Testbed: A Collaborative Design Studio for Field Robotics. *IEEE International Conference on Automation Science and Engineering*, v. 2021- August, p. 724–731, 23 ago. 2021.

BINDER.ORG. *Binder*. Disponível em: <<https://mybinder.org/>>. Acesso em: 21 jun. 2025.

BOWER, Matt. Technology-mediated learning theory. *British Journal of Educational Technology*, v. 50, n. 3, p. 1035–1048, 1 maio 2019.

BURGH-OLIVÁN, Miguel; ARAGÜÉS, Rosario; LÓPEZ-NICOLÁS, Gonzalo. *ROS-Based Multirobot System for Collaborative Interaction. Lecture Notes in Networks and Systems*. [S.l.: s.n.], 2023.

Disponível em: <<https://link.springer.com/bookseries/15179>>.

CASAÑ, Gustavo A. *et al.* ROS-based online robot programming for remote education and training. 29 jun. 2015, [S.l.]: Institute of Electrical and Electronics Engineers Inc., 29 jun. 2015. p. 6101–6106.

CERVERA, Enric; DEL POBIL, Angel P. ROSLab: Sharing ROS Code Interactively With Docker and JupyterLab. *IEEE Robotics and Automation Magazine*, v. 26, n. 3, p. 64–69, 1 set. 2019.

CHAUDHARY, Harsh. *Robot Operating System (ROS): The key to the future of robotics programming*. Disponível em: <<https://opencloudware.com/robot-operating-system-ros-the-key-to-the-future-of-robotics-programming/>>. Acesso em: 28 maio 2024.

CHITTA, Sachin; SUCAN, Ioan; COUSINS, Steve. MoveIt! *IEEE Robotics and Automation Magazine*, v. 19, n. 1, p. 18–19, mar. 2012.

DOCKER. *Docker Docs*. Disponível em: <<https://docs.docker.com/>>. Acesso em: 21 jun. 2025.

DOS SANTOS LOPES, Maisa Soares *et al.* Web environment for programming and control of a mobile robot in a remote laboratory. *IEEE Transactions on Learning Technologies*, v. 10, n. 4, p. 526–531, 1 out. 2017.

DYNAMICS BOSTON. *Navigating the Future of Robotics: Mastering ROS2 for Competitive Advantage*. Disponível em: <<https://blog.boston-engineering.com/navigating-the-future-of-robotics-mastering-ros2-for-competitive-advantage>>. Acesso em: 21 jun. 2025.

GAZEBO. *Gazebo: Tutorial: ROS overview*. Disponível em: <https://classic.gazebosim.org/tutorials?tut=ros_overview>. Acesso em: 17 jun. 2025.

GITHUB. *GitHub Codespaces · GitHub*. Disponível em: <<https://github.com/features/codespaces>>. Acesso em: 21 jun. 2025.

GONZÁLEZ-GARCÍA, Salvador *et al.* Designing a teaching guide for the use of simulations in undergraduate robotics courses: a pilot study. *International Journal on Interactive Design and Manufacturing*, v. 13, n. 3, p. 923–933, 1 set. 2019.

GUIMARÃES, Eliane *et al.* Design and implementation issues for modern remote laboratories. *IEEE Transactions on Learning Technologies*, v. 4, n. 2, p. 149–161, 2011.

HRANOV, Tsveti Hristov; HINOV, Nikolay Lyuboslavov. Open Source Robotics Platform for Educational Purposes. [S.d.].

KARALEKAS, Georgios; VOLOGIANNIDIS, Stavros; KALOMIROS, John. Europa: A case study for teaching sensors, data acquisition and robotics via a ROS-based educational robot. *Sensors (Switzerland)*, v. 20, n. 9, 2020a. Disponível em: <www.mdpi.com/journal/sensors>. Acesso em: 28 maio 2024.

KARALEKAS, Georgios; VOLOGIANNIDIS, Stavros; KALOMIROS, John. EUROPA: A Case Study for Teaching Sensors, Data Acquisition and Robotics via a ROS-Based Educational Robot. *Sensors* 2020, Vol. 20, Page 2469, v. 20, n. 9, p. 2469, 27 abr. 2020b. Disponível em: <<https://www.mdpi.com/1424-8220/20/9/2469/htm>>. Acesso em: 20 jun. 2025.

KHINE, Myint Swe. Robotics in STEM Education: Redesigning the Learning Experience. *Robotics in STEM Education: Redesigning the Learning Experience*, p. 1–262, 10 jul. 2017.

KRISTENSEN, Christoffer B. *et al.* Towards a robot simulation framework for E-waste disassembly using reinforcement learning. 2019, [S.l.: s.n.], 2019.

KRUMINS, Davis *et al.* Open Remote Web Lab for Learning Robotics and ROS with Physical and Simulated Robots in an Authentic Developer Environment. *IEEE Transactions on Learning Technologies*, v. 17, p. 1325–1338, 2024.

KRUTSCH, Emily; RODERICK, Victoria. *STEM Day: Explore Growing Careers | U.S. Department of Labor Blog*. Disponível em: <<https://blog.dol.gov/2022/11/04/stem-day-explore-growing-careers>>. Acesso em: 28 maio 2024.

KULICH, Miroslav *et al.* SyRoTek-Distance teaching of mobile robotics. *IEEE Transactions on Education*, v. 56, n. 1, p. 18–23, 2013.

LEE, Jeongseok *et al.* *DART: Dynamic Animation and Robotics Toolkit*. Disponível em: <<https://dartsim.github.io/>>. Acesso em: 28 maio 2024.

LI, Binghui; CHEN, Badong. An Adaptive Rapidly-Exploring Random Tree. *IEEE/CAA Journal of Automatica Sinica*, v. 9, n. 2, p. 283–294, 1 fev. 2022.

MARIAN, Marius *et al.* A ROS-based Control Application for a Robotic Platform Using the Gazebo 3D Simulator. *Proceedings of the 2020 21st International Carpathian Control Conference, ICC3 2020*, 27 out. 2020.

MARTIN, Sergio *et al.* The Future of Educational Technologies for Engineering Education. *IEEE Transactions on Learning Technologies*, v. 14, n. 5, p. 613–623, 1 out. 2021.

MEGALINGAM, Rajesh Kannan *et al.* ROS based Six-DOF robotic arm control through CAN bus interface. *Proceedings - 5th International Conference on Intelligent Computing and Control Systems, ICICCS 2021*, p. 739–744, 6 maio 2021.

MICHIELETTO, Stefano *et al.* Why teach robotics using ROS. *Journal of Automation, Mobile Robotics and Intelligent Systems*, p. 60–68, 22 jan. 2014.

NIEDŹWIECKI, Arthur *et al.* Cloud-Based Digital Twin for Cognitive Robotics. 2024, [S.l.]: IEEE Computer Society, 2024.

NOVNC. *noVNC*. Disponível em: <<https://novnc.com/info.html>>. Acesso em: 21 jun. 2025.

OPEN SOURCE ROBOTICS FOUNDATION. ROS: *Why ROS?* Disponível em: <<https://www.ros.org/blog/why-ros/>>. Acesso em: 21 jun. 2025.

PENG, Gang *et al.* Introduction to ROS 2 and Programming Foundation. *Introduction to Intelligent Robot System Design*, p. 541–566, 2023. Disponível em: <https://link.springer.com/chapter/10.1007/978-981-99-1814-0_11>. Acesso em: 17 jun. 2025.

PETROVI, Pavel; BALOGH, Richard. Deployment of Remotely-Accessible Robotics Laboratory. *International Journal of Online and Biomedical Engineering (iJOE)*, v. 8, n. S2, p. 31–35, 24 mar. 2012. Disponível em: <<https://online-journals.org/index.php/i-joe/article/view/1958>>. Acesso em: 21 jun. 2025.

PIETRZIK, S.; CHANDRASEKARAN, B. Setting up and Using ROS-Kinetic and Gazebo for Educational Robotic Projects and Learning. *Journal of Physics: Conference Series*, v. 1207, n. 1, 26 abr. 2019.

PITZER, Benjamin *et al.* PR2 Remote Lab: An environment for remote development and experimentation. *Proceedings - IEEE International Conference on Robotics and Automation*, p. 3200–3205, 2012.

PLAYER. *The Player Project - Free Software tools for robot and sensor applications*. Disponível em: <<https://playerstage.sourceforge.net/>>. Acesso em: 21 jun. 2025.

PROJECT JUPYTER. *Documentação oficial do Jupyter*. Disponível em: <<https://docs.jupyter.org/en/latest/>>. Acesso em: 20 jun. 2025.

PYBULLET. *Bullet Real-Time Physics Simulation | Home of Bullet and PyBullet: physics simulation for games, visual effects, robotics and reinforcement learning*. Disponível em: <<https://pybullet.org/wordpress/>>. Acesso em: 28 maio 2024.

QUIGLEY, Morgan *et al.* ROS: an open-source Robot Operating System. [S.d.]. Disponível em: <<http://stair.stanford.edu>>. Acesso em: 4 mar. 2024.

RICARDO TELLEZ. *A History of ROS (Robot Operating System)*. Disponível em: <<https://www.theconstruct.ai/history-ros/>>. Acesso em: 21 jun. 2025.

ROLDÁN-ÁLVAREZ, David *et al.* A ROS-based Open Web Platform for Intelligent Robotics Education. p. 243–255, 2022. Disponível em: <https://link.springer.com/chapter/10.1007/978-3-030-82544-7_23>. Acesso em: 25 maio 2023.

ROLDAN-ÁLVAREZ, David *et al.* Unibotics: open ROS-based online framework for practical learning of robotics in higher education. *MULTIMEDIA TOOLS AND APPLICATIONS*, abr. 2023.

ROLDÁN-ÁLVAREZ, David *et al.* Unibotics: open ROS-based online framework for practical learning of robotics in higher education. *Multimedia Tools and Applications*, p. 1–26, 14 nov. 2023. Disponível em: <<https://link-springer-com.ez88.periodicos.capes.gov.br/article/10.1007/s11042-023-17514-z>>. Acesso em: 5 abr. 2024.

ROS. *ROS/Concepts - ROS Wiki*. Disponível em: <<https://wiki.ros.org/ROS/Concepts>>. Acesso em: 21 jun. 2025a.

ROS. *rviz - ROS Wiki*. Disponível em: <<https://wiki.ros.org/rviz>>. Acesso em: 16 jun. 2025b.

ROS - WIKI. *ROS: Home*. Disponível em: <<https://www.ros.org/>>. Acesso em: 28 maio 2024.

RTI COMMUNITY. *ROS 2: What is DDS | Data Distribution Service (DDS) Community RTI Connex* Users. Disponível em: <<https://community.rti.com/page/ros-2-what-dds>>. Acesso em: 21 jun. 2025.

SHERMAN, Michael; EASTMAN, Peter. *SimTK: Simbody: Multibody Physics API: Project Home*. Disponível em: <<https://simtk.org/projects/simbody/>>. Acesso em: 28 maio 2024.

SHORT, Andrew *et al.* Recent progress on sampling based dynamic motion planning algorithms. *IEEE/ASME International Conference on Advanced Intelligent Mechatronics, AIM*, v. 2016- Septe, p. 1305–1311, 2016.

SOFTWARE, Torus Knot. *OGRE - Open Source 3D Graphics Engine \- Home of a marvelous rendering engine*. Disponível em: <<https://www.ogre3d.org/>>. Acesso em: 28 maio 2024.

SUCAN, I A; CHITTA, S. *MoveIt Motion Planning Framework*. Disponível em: <<http://moveit.ros.org/about/>>. Acesso em: 28 maio 2024.

TAKAYA, Kenta *et al.* Simulation environment for mobile robots testing using ROS and Gazebo. 2016 *20th International Conference on System Theory, Control and Computing, ICSTCC 2016 - Joint Conference of SINTES 20, SACCS 16, SIMSIS 20 - Proceedings*, p. 96–101, 16 dez. 2016.

TELLEZ, Ricardo. A thousand robots for each student: Using cloud robot simulations to teach robotics. *Advances in Intelligent Systems and Computing*, v. 457, p. 143–155, 2017. Disponível em: <https://link.springer.com/chapter/10.1007/978-3-319-42975-5_14>. Acesso em: 1 abr. 2025.

THANGS. *Robotic Arm 3D Model.STEP - 3D model by singh.vikas5081 on Thangs*. Disponível em: <[https://thangs.com/designer/singh.vikas5081/3d-model/Robotic Arm 3D Model.STEP-198882](https://thangs.com/designer/singh.vikas5081/3d-model/Robotic%20Arm%203D%20Model.STEP-198882)>. Acesso em: 20 jun. 2025.

VENTUZELOS, Vítor; LEÃO, Gonçalo; SOUSA, Armando. Teaching ROS1/2 and Reinforcement Learning using a Mobile Robot and its Simulation. *Lecture Notes in Networks and Systems*, v. 589 LNNS, p. 586–598, 2023. Disponível em: <https://doi.org/10.1007/978-3-031-21065-5_48>. Acesso em: 1 abr. 2025.

VERNER, Igor M.; CUPERMAN, Dan; REITMAN, Michael. Exploring Robot Connectivity and Collaborative Sensing in a High-School Enrichment Program. *Robotics 2021, Vol. 10, Page 13*, v. 10, n. 1, p. 13, 7 jan. 2021. Disponível em: <<https://www.mdpi.com/2218-6581/10/1/13/htm>>. Acesso em: 28 maio 2024.

VISUAL STUDIO CODE. *Visual Studio Code - Code Editing. Redefined*. Disponível em: <<https://code.visualstudio.com/>>. Acesso em: 21 jun. 2025.

VROCHIDOU, Eleni *et al.* Open-source robotics: Investigation on existing platforms and their application in education. 2018 *26th International Conference on Software, Telecommunications and Computer Networks, SoftCOM 2018*, v. 2018- January, 30 nov. 2018.

WIEDMEYER, Wolfgang *et al.* Robotics education and research at scale: A remotely accessible robotics development platform. *Proceedings - IEEE International Conference on Robotics and Automation*, v. 2019- May, p. 3679–3685, 1 maio 2019.

XFCE. *Ambiente de trabalho Xfce*. Disponível em: <<https://www.xfce.org/>>. Acesso em: 21 jun. 2025.

ANEXOS

1. Arquivos de configuração do Docker contêiner

Arquivo: dockerfile

```
1 FROM osrf/ros:noetic-desktop-full
2
3 # === CONFIGURAÇÃO DE USUÁRIO NÃO-ROOT ===
4 ARG USERNAME=ubuntu
5 ARG USER_UID=1000
6 ARG USER_GID=$USER_UID
7
8 ENV DEBIAN_FRONTEND=noninteractive
9
10 # === INSTALA FERRAMENTAS BÁSICAS E REMOVE REPOSITÓRIO ANTIGO DO ROS ===
11 RUN [ -f /etc/apt/sources.list.d/ros1-latest.list ] && sed -i '/packages.ros.org/d' /etc/apt/sources.list.d/ros1-latest.list || true \
12     && apt-get update \
13     && apt-get install -y curl wget gnupg2 lsb-release ca-certificates sudo
14
15 # === INSTALA NOVA CHAVE GPG DO ROS ===
16 RUN curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc \
17     | gpg --dearmor -o /usr/share/keyrings/ros-archive-keyring.gpg
18
19 # === RECONFIGURA O REPOSITÓRIO DO ROS ===
20 RUN echo "deb [signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" \
21     > /etc/apt/sources.list.d/ros1-latest.list
22
23 # === CRIA USUÁRIO ===
24 RUN groupadd --gid $USER_GID $USERNAME && \
25     useradd -s /bin/bash --uid $USER_UID --gid $USER_GID -m $USERNAME && \
26     usermod -aG sudo,dialout,video $USERNAME && \
27     echo "$USERNAME ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/$USERNAME && \
28     chmod 0440 /etc/sudoers.d/$USERNAME
29
30 USER $USERNAME
31
32 # === ATUALIZA SISTEMA E INSTALA DEPENDÊNCIAS DO ROS ===
33 RUN sudo apt update && sudo apt upgrade -y && \
34     sudo apt install -y git nano python-is-python3 \
35     ros-noetic-ros-tutorials \
36     ros-noetic-gazebo-ros-pkgs ros-noetic-gazebo-ros-control \
37     liburdfdom-tools \
38     ros-noetic-effort-controllers \
39     ros-noetic-ros-control ros-noetic-ros-controllers \
40     ros-noetic-rviz-visual-tools \
41     python3-catkin-tools \
42     ros-noetic-moveit \
43     ros-noetic-catkin \
44     ros-noetic-serial \
45     ros-noetic-rqt-joint-trajectory-controller \
46     libpoco-dev \
47     python3-wstool
48
49 # === CONFIGURA ROSDEP (COMO ROOT) ===
50 USER root
51 RUN rosdep init && \
52     rosdep update || true
53
54 # === INSTALA XFCE + VNC ===
55 RUN apt-get update && \
56     apt-get install -y --no-install-recommends \
57     xfce4 xfce4-terminal \
58     novnc websockify xterm x11vnc net-tools \
59     python3-pip libgl1-mesa-dri libgl1-mesa-glx dbus-x11 mesa-utils \
60     x11-utils x11-xserver-utils wmctrl xdotool firefox autocutsel && \
61     apt-get clean
```

```

63 # === CONFIGURA DISPLAY ===
64 ENV DISPLAY=:1
65
66 # === DESABILITA AVISO DE EOL DO ROS ===
67 ENV DISABLE_ROS1_EOL_WARNINGS=1
68
69 # === DEPENDÊNCIAS DO VSCODE ===
70 RUN apt-get update && apt-get install -y \
71     gnome-keyring \
72     libsecret-1-0 libsecret-1-dev
73
74 # === INSTALA TURBOVNC ===
75 RUN curl -L -o /tmp/turbovnc.deb https://github.com/TurboVNC/turbovnc/releases/download/3.1.1/turbovnc_3.1.1_amd64.deb && \
76     dpkg -i /tmp/turbovnc.deb || apt-get install -fy && \
77     rm /tmp/turbovnc.deb
78
79 ENV PATH="/opt/TurboVNC/bin:${PATH}"
80
81 # === INSTALA VSCODE ===
82 RUN curl -fsSL https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor -o /etc/apt/trusted.gpg.d/microsoft.gpg && \
83     echo "deb [arch=amd64] https://packages.microsoft.com/repos/vscode stable main" > /etc/apt/sources.list.d/vscode.list && \
84     apt update && apt install -y code
85
86 # === CONFIGURA VNC ===
87 RUN mkdir -p /home/${USERNAME}/.vnc && \
88     echo "senha-segura" | /opt/TurboVNC/bin/vncpasswd -f > /home/${USERNAME}/.vnc/passwd && \
89     chmod 600 /home/${USERNAME}/.vnc/passwd && \
90     chown -R ${USERNAME}:${USERNAME} /home/${USERNAME}/.vnc
91
92 # === COPIA SCRIPTS ===
93 COPY ./scripts/*.sh /home/${USERNAME}/
94 RUN chown ${USERNAME}:${USERNAME} /home/${USERNAME}/*.sh && \
95     chmod +x /home/${USERNAME}/*.sh
96
97 # === CONFIGURA ROS NO BASHRC ===
98 RUN echo "source /opt/ros/noetic/setup.bash" >> /home/${USERNAME}/.bashrc && \
99     echo "source /projeto1_PosDoc/init.bash" >> /home/${USERNAME}/.bashrc
100
101 # === COPIA O WORKSPACE ===
102 COPY catkin_ws /home/${USERNAME}/catkin_ws
103
104 RUN echo "source /home/${USERNAME}/catkin_ws/devel/setup.bash" >> /home/${USERNAME}/.bashrc && \
105     sudo chown -R $USERNAME:$USERNAME /home/${USERNAME}/catkin_ws
106
107 # === SCRIPT: ABRE TUTORIAL + RVIZ ===
108 RUN mkdir -p /home/${USERNAME}/Desktop && \
109     echo '#!/bin/bash\n\
110     source /opt/ros/noetic/setup.bash\n\
111     source /home/ubuntu/catkin_ws/devel/setup.bash\n\
112     RES=$(xdpyinfo | grep dimensions | awk "{print \$2}")\n\
113     WIDTH=$(echo $RES | cut -d "x" -f1)\n\
114     HEIGHT=$(echo $RES | cut -d "x" -f2)\n\
115     HALF_WIDTH=$(( $WIDTH / 2 ))\n\
116     xdg-open "https://moveit.github.io/moveit_tutorials/doc/quickstart_in_rviz/quickstart_in_rviz_tutorial.html" &\n\
117     sleep 3\n\
118     wmctrl -r :ACTIVE: -e 0,0,0,$HALF_WIDTH,$HEIGHT\n\
119     xfce4-terminal --title="RViz Tutorial" --geometry=100x30 --hold -e "bash -c \"roslaunch panda_moveit_config demo.launch rviz_tutorial:=true\" \"\" &\n\
120     sleep 5\n\
121     wmctrl -r RViz -e 0,$HALF_WIDTH,0,$HALF_WIDTH,$HEIGHT\n\
122     > /home/${USERNAME}/start_tutorial_and_rviz.sh && \
123     chmod +x /home/${USERNAME}/start_tutorial_and_rviz.sh && \
124     chown $USERNAME:$USERNAME /home/${USERNAME}/start_tutorial_and_rviz.sh
125
126 # === ATALHOS NA DESKTOP ===
127 RUN echo '[Desktop Entry]\n\
128 Version=1.0\n\
129 Type=Application\n\
130 Name=Init ROS\n\
131 Comment=Inicializa ambiente ROS\n\
132 Exec=/home/ubuntu/init_ros.sh\n\
133 Icon=utilities-terminal\n\
134 Terminal=true\n\
135 Categories=Development;' \
136 > /home/ubuntu/Desktop/init_ros.desktop && \
137 chmod +x /home/ubuntu/Desktop/init_ros.desktop && \
138 chown ubuntu:ubuntu /home/ubuntu/Desktop/init_ros.desktop
139
140 RUN echo '[Desktop Entry]\n\
141 Version=1.0\n\
142 Type=Application\n\
143 Name=Tutorial + RViz\n\
144 Comment=Abre navegador com o tutorial e o RViz lado a lado\n\
145 Exec=/home/ubuntu/start_tutorial_and_rviz.sh\n\
146 Icon=applications-internet\n\
147 Terminal=false\n\
148 Categories=Development;' \
149 > /home/ubuntu/Desktop/tutorial_and_rviz.desktop && \
150 chmod +x /home/ubuntu/Desktop/tutorial_and_rviz.desktop && \
151 chown ubuntu:ubuntu /home/ubuntu/Desktop/tutorial_and_rviz.desktop

```

```

153 # === WALLPAPER ===
154 COPY ../images/wallpapers/ /usr/share/backgrounds/labvir/
155 RUN chmod 644 /usr/share/backgrounds/labvir/*
156
157 USER $USERNAME
158 RUN mkdir -p /home/$USERNAME/.config/autostart && \
159     echo "[Desktop Entry]\n\
160     Type=Application\n\
161     Name=Set Wallpaper\n\
162     Exec=/home/ubuntu/set-wallpaper.sh\n\
163     Terminal=false\n\
164     X-GNOME-Autostart-enabled=true" \
165     > /home/$USERNAME/.config/autostart/set-wallpaper.desktop && \
166     chmod +x /home/$USERNAME/.config/autostart/set-wallpaper.desktop
167
168 # === INSTALA ROS-APT-SOURCE (MANTÉM AS CHAVES ATUALIZADAS) ===
169 USER root
170 RUN apt update && apt install -y ros-apt-source
171
172 # === PORTAS E COMANDO PADRÃO ===
173 EXPOSE 5901 6080
174 USER $USERNAME
175 CMD ["/home/ubuntu/start-vnc.sh"]

```

Arquivo: devcontainer.json

```

1 // Desabilita a validação do schema para evitar erros HTTP 429
2 {
3     // URL do schema do devcontainer (opcional, para validação de estrutura)
4     "$schema": "https://raw.githubusercontent.com/devcontainers/spec/main/schemas/devContainer.schema.json",
5
6     // Nome exibido no VSCode para este container
7     "name": "noetic_ros_lab1",
8
9     // Dockerfile que será usado para construir a imagem do container
10    "dockerFile": "Dockerfile",
11
12    // Contexto de build do Docker (aqui é o diretório pai "..")
13    "context": "..",
14
15    // Argumentos adicionais para o Docker Run
16    "runArgs": [
17        "--privileged", // Permite acesso privilegiado (necessário para alguns recursos, como dispositivos USB, redes, etc.)
18        "--env=DISPLAY", // Exporta a variável DISPLAY para suportar aplicações gráficas X11
19        "--volume=/tmp/.X11-unix:/tmp/.X11-unix:rw", // Monta o socket X11 para forwarding gráfico (Linux → container)
20        "--shm-size=2g" // Aumenta o tamanho da memória compartilhada (shared memory) para evitar problemas em aplicativos como o RViz ou Gazebo
21    ],
22
23    // Portas expostas do container para o host
24    "forwardPorts": [
25        6080, // Porta do noVNC (desktop no navegador)
26        5901 // Porta do servidor VNC (acesso direto via cliente VNC)
27    ],

```



```

29 // Atributos personalizados para cada porta
30 "portsAttributes": {
31   "6080": {
32     "label": "noVNC Desktop", // Nome exibido no VSCode
33     "onAutoForward": "openBrowser" // Ao rodar, abre automaticamente no navegador
34   },
35   "5901": {
36     "label": "VNC Desktop", // Nome da porta
37     "onAutoForward": "notify" // Apenas notifica que a porta está disponível (não abre automaticamente)
38   }
39 },
40
41 // Montagem do workspace (diretório local) no container
42 "workspaceMount": "source=${localWorkspaceFolder},target=${localWorkspaceFolderBasename},type=bind",
43
44 // Diretório de trabalho dentro do container
45 "workspaceFolder": "${localWorkspaceFolderBasename}",
46
47 // Montagens adicionais
48 "mounts": [
49   // Monta o histórico de comandos do bash do usuário local dentro do container
50   "source=${localEnv:HOME}${localEnv:USERPROFILE}/.bash_history,target=/home/vscode/.bash_history,type=bind"
51 ],
52
53 // Customizações no VSCode dentro do devcontainer
54 "customizations": {
55   "vscode": {
56     "extensions": [
57       "ms-python.python", // Suporte ao Python
58       "ms-vscode.cpptools", // Suporte ao C/C++
59       "ms-iot.vscode-ros", // Extensão oficial do ROS para VSCode
60       "ms-vscode-remote.remote-containers", // Integração do VSCode com containers
61       "ms-azuretools.vscode-docker", // Extensão para gerenciamento de containers Docker
62       "github.vscode-pull-request-github", // Extensão GitHub (PRs, Issues, etc.)
63       "mdickin.markdown-shortcuts", // Atalhos para Markdown
64       "sourcegraph.cody-ai", // Assistente de IA Cody (opcional)
65       "davidanson.vscode-markdownlint" // Linter de Markdown (boas práticas e formatação)
66     ]
67   }
68 },
69
70 // Features pré-configuradas disponibilizadas pela comunidade Devcontainers
71 "features": {
72   // Pacote de utilitários comuns
73   "ghcr.io/devcontainers/features/common-utils:2": {
74     "installZsh": true, // Instala Zsh
75     "installOhMyZsh": true, // Instala Oh My Zsh (tema e plugins)
76     "upgradePackages": true // Atualiza pacotes do sistema na criação
77   },
78
79   // Suporte ao desktop gráfico (XFCE) com VNC e noVNC
80   "ghcr.io/devcontainers/features/desktop-lite:1": {
81     "desktop": "xfce", // Escolhe XFCE como ambiente gráfico
82     "useVnc": true, // Habilita servidor VNC + noVNC (acesso via navegador ou cliente VNC)
83     "vncPassword": "", // Senha do VNC (se vazia, normalmente não exige ou usa default como 'vscode')
84     "installExtensions": true, // Instala automaticamente as extensões do VSCode listadas
85     "omitDesktopApps": true // Omite instalação de aplicativos de desktop não essenciais (libera espaço)
86   }
87 }
88 }

```

Arquivo: start-vnc.sh

```
1  #!/bin/bash
2
3  # Configurações de ambiente
4  export USER=${USER:-ubuntu}
5  export HOME=/home/$USER
6  export DISPLAY=:1
7
8  # Mata instâncias antigas e limpa arquivos temporários
9  vncserver -kill $DISPLAY >/dev/null 2>&1 || true
10 rm -rf /tmp/.X1-lock /tmp/.X11-unix/X1 $HOME/.vnc/*
11
12 # Cria xstartup para o XFCE
13 mkdir -p "$HOME/.vnc"
14 cat <<EOF > "$HOME/.vnc/xstartup.turbovnc"
15 #!/bin/sh
16 autocutsel -fork
17 unset SESSION_MANAGER
18 exec startxfce4
19 EOF
20 chmod +x "$HOME/.vnc/xstartup.turbovnc"
21
22 # Detecta resolução atual (ou usa fallback)
23 CURRENT_RESOLUTION=$(xrandr | grep '*' | awk '{print $1}' | head -n1)
24 if [ -z "$CURRENT_RESOLUTION" ]; then
25     CURRENT_RESOLUTION="1920x900"
26 fi
27 WIDTH=$(echo $CURRENT_RESOLUTION | cut -d'x' -f1)
28 HEIGHT=$(echo $CURRENT_RESOLUTION | cut -d'x' -f2)
29
30 echo "Resolução detectada: ${WIDTH}x${HEIGHT}"
31
32 # Inicia Xvfb (necessário em ambientes headless)
33 echo "Iniciando Xvfb no display $DISPLAY..."
34 Xvfb $DISPLAY -screen 0 ${WIDTH}x${HEIGHT}x24 +extension GLX +render -noreset >/dev/null 2>&1 &
35
36 sleep 2
37
38 # Inicia o VNC
39 echo "Iniciando TurboVNC no display $DISPLAY..."
40 vncserver $DISPLAY -geometry ${WIDTH}x${HEIGHT} -depth 24 \
41     -xstartup "$HOME/.vnc/xstartup.turbovnc" \
42     -securitytypes TLSNone \
43     -noxstartup
44
45 # Verifica se o VNC subiu corretamente
46 if ss -tuln | grep ":5901" >/dev/null; then
47     echo "✅ VNC ativo na porta 5901."
48 else
49     echo "❌ VNC NÃO está rodando na porta 5901. Abortando."
50     exit 1
51 fi
52
53 # Inicia o websockify se não estiver rodando
54 if ! pgrep -f "websockify.*6080" > /dev/null; then
55     echo "Iniciando websockify em :6080..."
56     websockify --web /usr/share/novnc 6080 localhost:5901 >/dev/null 2>&1 &
57 else
58     echo "websockify já está rodando. Pulando."
59 fi
60
61 # Checa se o DISPLAY está ativo
62 export DISPLAY=:1
63 xdpinfo >/dev/null 2>&1 && echo "DISPLAY ok!" || echo "Falha ao acessar DISPLAY"
64
65 # Aguarda XFCE subir
66 timeout 20s bash -c 'while ! pgrep -f xfce4-session >/dev/null; do sleep 1; done'
67
68 echo "XFCE carregado com sucesso."
```

2. Estruturas e links para os repositórios do projeto

Homepage do projeto - <https://github.com/NURIA-IFSP/website>

➤ ROS 101

- Tutoriais:
<https://github.com/NURIA-IFSP/ROS-Tutorials-Jupyter>
https://github.com/NURIA-IFSP/ros_tutorials
- Demonstrações:
<https://github.com/NURIA-IFSP/atividade-01-introducao-ao-ROS>
<https://github.com/NURIA-IFSP/atividade-02-introducao-ao-ROS>
<https://github.com/NURIA-IFSP/atividade-03-introducao-ao-ROS>
- Projeto - *TurtleParty*
https://github.com/NURIA-IFSP/ros101_projeto_turtleparty

➤ Manipulação robótica 101

- Tutoriais:
https://github.com/NURIA-IFSP/LabVir_moveit_tutorial
- Demonstrações:
https://github.com/NURIA-IFSP/LabVir_moveit_ur10_lab1/
https://github.com/NURIA-IFSP/LabVir_moveit_ur10_lab1_new
- Projeto -
https://github.com/NURIA-IFSP/LabVir_moveit_Lab1_v1.2_projeto
https://github.com/NURIA-IFSP/LabVir_moveit_ur5_lab2

➤ Laboratórios Virtuais

- ROS 1 – Noetic Ninjemys:
https://github.com/NURIA-IFSP/LabVir_noetic_lab
- ROS 2 – Jazzy Jalisco:
https://github.com/NURIA-IFSP/LabVir_noetic_lab

➤ Exercícios e atividades

- Introdução ao ROS:
https://github.com/NURIA-IFSP/LabVir_Tutorial_ROS_atividades_Completo